How I met Claude Code

A practical guide to coding agents



V0.2.1 – April 2026

To Zoé

My little un-artificial slice of life

TABLE OF CONTENTS

Preface 11

Chapter 1: Before Claude Code — The GPT Era 15

- The First Agent 16
- The Vibe Coding Origin Story 17
- The Surprise Discovery 18
- What AI Looked Like at Work 19
- Other Pre-Claude Projects 20
- The Pattern I Couldn't See Yet 21

Chapter 2: December 2025 — First Contact 23

- The Strategic Awakening 23
- The Blog Posts Begin 25
- Claude Analyzes My Voice 27
- Something Was Shifting 30

Chapter 3: The Philosopher Tools 33

- From Skills to Tools 33
- The Army of Philosophers 34
- Why Philosophers? 36
- The Zarathustra Project 38
- The Philosopher Rap Battle 38
- The Developer Workflow 39
- Tools as Self-Portraits 41

Chapter 4: The Competitive Intelligence Machine 43
The Pattern 43
The PIDO Deep Dive 45
The Neuroscience Toolbox Analysis 46
MATLAB Community Intelligence 47
Conference Intelligence 48
What Claude Brought, What I Brought 49
The Sleepless Nights 50
The Quality Question 51
What This Changed 52

Chapter 5: The MCP Standardization 55
USB-C for AI 55
The MATLAB MCP Core Server 56
The Integration Play 58
The MCP Server Experiments 59
The Blog Post Machine 60
The Onboarding Campaign 61
Skills Get a Home 62
Agents Calling Agents 64
What MCP Actually Changed 65

Chapter 6: The Side Projects That Kept Growing 67
recettesdezoe.com — From Demo to Business? 68
The Mom Test 69
The Competitive Landscape 69
The Voice Test 70

Content Scaling with Zoé 71
The YouTube Channel That Became a Lab 72
Tennis — Predicting the French Open 72
NBA — Dashboards and Automation 73
NFL — From Toolbox to Clutch Factor 73
Cycling — The Ghost That Kept Running 75
Soccer — The Premier League Dashboard 75
The Pattern 77
Gaining Back Control Over the Algorithm 77
The Architecture 78
Form Factors 79
Use Cases That Multiplied 80
The Philosophical Resonance 80
The Portfolio Effect 81
What the Side Projects Taught Me 83

Chapter 7: Writing the Future — Sharing Knowledge 87
The Presentation That Kept Evolving 87
Outbound Marketing — The Workflow Version 88
Industry Marketing — The Technically Ambitious Version 89
Educator Programs — The One Built from Scratch 90
The Learning Curve in Four Weeks 90
The Live Demo That Changed Minds 93
The Blog Post Feedback Loop 95
The Book Club — Co-Intelligence as Mirror 96
Resistance and Real Concerns 98
From User to Advocate 100

Chapter 8: The Permanent Resident 103

What ClawdBot Was 103

The Moment That Spooked Me 105

The Rebrand, and What It Said 106

The Resurrection 107

What Jean-Clawd Actually Does 108

Why This Is Different from Claude Code for the Web 109

Claude Code Catching Up 110

The AND, Again 111

Chapter 9: What This Relationship Actually Is 115

The Five Modes 115

What Makes This Different 117

The Identity Question 118

Productive Anthropomorphism 119

The Honest Accounting 120

Where This Goes 122

PREFACE

"The proper study of man is anything but himself, and it is through the study of language that we learn about our own nature." – J.R.R. Tolkien

Language is a foundation for humanity. It is through language that we have elevated ourselves, from the silence of nature. We shared with one another. We started telling each other stories, sometimes simply relaying information, sometimes making things up. We elaborated ideas, started reasoning about them, turning them into plans. A sound became words — words transformed into writings, writings into books, books into cultures, cultures into civilizations. Leading us all the way to the current digital age, with the inter-connected net of articles, blogs, images, videos. All this content that started to feed machines, eager to learn.

In my previous book, *Programming GPTs*, I wrote about how artificial intelligence had marked a new epoch in our relationship with language — how models built on the crunching of numbers by the largest supercomputers in the world had enabled computers to understand and generate human language with unprecedented accuracy. I described the next wave that was gradually taking the human away from the conversational loop: AI agents. Agents that build on large language models, but that can act autonomously, using tools and carrying out entire workflows without necessarily needing tight supervision.

That book was designed to equip readers with the knowledge to harness the power of agents. This book is something else entirely. This one is personal.

As I was about to respond to a friend who had asked if there was a GPT to look up flight prices to Boston, it got me to reflect on the journey of AI for the past few years. GPTs were a thing — I had written a whole book about them — and then Claude Code arrived. And something shifted. Not just in the technology, but in the relationship. The chatbot I had studied from a distance became a collaborator sitting beside me in the terminal.

This is the story of how I met Claude. But mainly Claude Code.

It was December of 2025. For some reason, this is when the world started to pay attention to Claude Code, and what was happening with coding agents. The company I work for (the MathWorks, developers of MATLAB) was starting a pilot to assess the potential for non-developers first. A few product managers were given access, with the responsibility to report back on their usage in the first quarter of 2026. I was one of the lucky ones.

What I didn't expect was that a three-month pilot would turn into a book. That a coding assistant would become a thinking partner. That I would find myself, night after night, building philosophy websites, competitive intelligence tools, cooking apps, and sports analytics channels — not because anyone asked me to, but because for the first time, the gap between having an idea and making it real had nearly vanished.

This is not a technical manual. It is a memoir of collaboration — an honest account of what happens when a product manager with more curiosity than coding skill meets an AI that doesn't just answer questions, but builds alongside you.

Chapter 1: Before Claude Code — The GPT Era

By mid-2025, you were probably deep in the GPT era — like me — and didn't even realize a seismic shift was coming.

Everyone I knew was experimenting. Product managers were tip-toeing with the technology, prototyping with AI services or locally with Ollama. Marketing teams were generating content drafts. Developers were starting to evaluate copilots and some were happy enough about the productivity gain it was providing. It's in mid-2025 that the term "vibe coding" entered the vocabulary — this idea that you could describe what you wanted in plain language and watch an AI conjure it into existence. It felt like magic, and like most magic, we didn't fully understand the trick.

I was spending all the free hours I had on AI. I had my ChatGPT subscription, my collection of apps connecting to the OpenAI web services, and a growing sense that something fundamental was changing about how software would be built. But in practice, working with AI meant a lot of friction. You'd generate something useful, copy it into another tool, realize the context was lost, paste it back, re-explain what you were trying to do. Every session started fresh. Every project lived in its own bubble. The AI had no memory

of you, no awareness of what you'd built together before. It was capable and frustrating in equal measure — like working with a brilliant colleague who had amnesia.

That was about to change. But the first crack in that model didn't come from Claude — it came on Valentine's Day, 2025, from an unlikely source.

The First Agent

On February 14th, 2025 — Valentine's Day, of all days — Perplexity launched Deep Research. I remember the date because it felt like a gift, though not the kind you'd expect.

Up until then, AI was something you talked *to*. You asked a question, it answered. You wrote a prompt, it generated text. The mental model was a very smart search box, or a very patient assistant who never got tired of your follow-up questions. The loop was always: me type, AI respond, me copy, me paste, me think.

Deep Research broke that loop. You gave it a question — a real, meaty one — and it went away. It searched. It read. It synthesized across dozens of sources. It came back with something that looked less like a chatbot response and more like a document a junior analyst might produce after a week of work. The first time I used it for a competitive analysis at MathWorks, I had that specific feeling that something had shifted: I hadn't helped it do research. It had done research, and showed me the results.

A colleague of mine had a reaction that stayed with me — watching someone replicate months of his market

analysis work with a single deep research prompt in under thirty minutes. I'll come back to that story in Chapter 7, when the book club puts a name to the feeling. But the seed of it was here, in early 2025, the first time a research task felt genuinely delegatable.

Still, there was a ceiling. Deep Research was powerful for synthesis, but it remained a one-shot interaction. You asked, it answered. There was no memory between sessions, no ongoing project, no sense of working *with* something over time. The AI was a very capable contractor you hired for a single job and never saw again. Next week, when the follow-up question came, you'd have to re-hire, re-brief, and re-explain everything from scratch — hoping the output would be consistent with what came before.

I didn't know yet what continuity would feel like. But I was about to get a taste of something closer to it.

The Vibe Coding Origin Story

Late August 2025. My in-laws were visiting, shortly after GPT-5's release. My father-in-law and I were in the kitchen late at night — the kind of hour where ambitious ideas feel perfectly reasonable.

Me: "What website or web app would you like to develop?" **Father-in-law**: *looks around the kitchen, spots a recipe book* **Me**: "Ok, let's build a website with French recipes."

Single prompt. Single session. I opened ChatGPT, described what I wanted — a clean, browsable website of French recipes — and watched it generate the

HTML, CSS, and JavaScript. The whole thing took maybe an hour, most of which was us discussing which recipes to feature and laughing about the absurdity of what we were doing.

That same night, sometime around 10pm, I purchased the domain on Cloudflare. recettesdezoe.com was born — named after my daughter Zoé. We deployed it right there, my father-in-law watching over my shoulder as the site went live.

It was a demo. A party trick. A thing I'd built to show what was possible.

Or so I thought.

The Surprise Discovery

Four months later — December 2025 — I was poking around my Cloudflare dashboard for an unrelated reason when something caught my eye. The analytics for recettesdezoe.com showed **1,300+ visitors in the past month.**

I stared at the number. Zero edits since launch. Zero marketing. Zero SEO work. Zero social media promotion. Nothing. I had literally forgotten the site existed for weeks at a time, and over a thousand people had found it, visited it, and presumably cooked something from it.

This broke something in my mental model. I had always assumed that building a useful product required sustained effort — iteration, marketing, user research, the whole product management playbook I'd internalized over years in my job. But here was a

website I'd built in a kitchen at 10pm with AI, and it was generating more organic traffic than projects I'd spent months planning.

The lessons hit me one at a time over the following days:

- **Vibe coding can produce genuinely useful, discoverable products.** Not just demos — real things that real people find and use.

- **The "demo that becomes real" pattern is more powerful than planned projects.** When you remove the friction of building, you can test ideas at the speed of conversation.

- **Domain purchases at 10pm sometimes pay off.** I'm not saying it's a strategy. But I'm not saying it isn't.

What AI Looked Like at Work

In September 2025, a month after the kitchen demo, I gave a presentation to the broader product marketing department at MathWorks with my colleague Jacob. We called it "GenAI Tools for Modern Tech Marketers." The focus was on Microsoft Copilot — the AI tools most widely available inside the organization — and we structured it around three personas: the product manager, the developer, and the outbound marketer.

For product managers, two tools stood out. The first was the Researcher — Microsoft's version of the deep research capability that ChatGPT and Perplexity had introduced earlier that year. You could ask it to analyze

a market, summarize a competitive landscape, synthesize a body of documentation, and it would produce something genuinely useful. The second was the Analyst — a data analysis tool integrated directly into the Microsoft 365 environment, with access to your Excel files. I used it to slice product usage data across dimensions that would have taken me hours to do manually.

The audience was receptive. Some people in the room had their own "sleepless night" moment during the session — a first encounter with AI that hit differently than they expected. We had built something that felt like real progress.

And it was. But preparing that presentation also clarified something I hadn't fully articulated before: every one of these tools operated in isolation. The Researcher didn't know what the Analyst had found. Neither of them knew what project you were working on, what decisions you were trying to make, or what you'd already tried. You stitched the outputs together manually, in your own head, across a dozen open tabs. The AI was useful in each individual moment and absent everywhere in between.

That was the state of the art in the enterprise in late 2025. It felt like progress — until you started wondering what it would feel like if the AI actually stuck around.

Other Pre-Claude Projects

recettesdezoe.com wasn't my only experiment in the pre-Claude era. That summer and fall, I'd been building

a small portfolio of AI-assisted projects, each one pushing a little further into what was possible.

GhostRunners.app was probably the most ambitious. I was getting back on my bicycle, and I wanted to get better at it: so i looked at analytics that could compare my current ride to previous efforts on the same route — essentially racing against my own ghost, like in a video game. I built it using MATLAB Mobile for data capture, MATLAB Online for the analysis, and Python with the Garmin FIT SDK for parsing the ride files. I even recorded YouTube videos documenting the process on my channel @yannspeaksai. It wasn't pretty code, but it worked, and it scratched an itch and motivated me to exercise more.

OpenAI Codex came next — first as a VSCode integration where I could vibe-code small HTML and JavaScript apps, and later as a cloud-based agent that could run data analysis hands-off. The progression was revealing: from AI helping me write code, to AI writing code while I watched, to AI writing and running code while I did something else entirely.

OpenAI Canvas I tried once and largely abandoned. For single-page web apps, I found myself defaulting to the Claude app — either on mobile or desktop. Even before Claude Code entered the picture, something about Claude's approach to Artifacts was pulling me in.

The Pattern I Couldn't See Yet

Looking back, these projects trace an arc I couldn't see at the time. Each one was a single interaction — a prompt, a session, an output. I'd build something, feel the rush of creation, and move on. There was no

continuity between sessions, no shared context, no sense that the AI remembered or built upon what we'd done before. I remember coming back to GhostRunners two weeks after building it, wanting to add a feature, and spending the first twenty minutes just re-explaining what the project was, what the data structure looked like, and what I'd already tried. The AI was helpful — but it was meeting me for the first time, again.

Every conversation started from zero. Every project was an island.

I didn't know it yet, but I was about to discover what happens when AI stops being a tool you pick up and put down, and starts becoming something closer to a collaborator who remembers where you left off.

December 2025 was coming, and with it, an invitation to a pilot program that would change how I thought about all of this.

Chapter 2: December 2025 — First Contact

The email arrived sometime in early December. MathWorks was running a pilot program — Claude Code for non-developers. A handful of product managers would get access, with one responsibility: report back on what they found in Q1 2026. I was one of the lucky early testers.

But I didn't start with Claude Code. That would have been too big a leap for someone coming from the ChatGPT world. Instead, I started where most people would: Claude Desktop.

The Desktop app is a very approachable interface for those who come from chat apps like ChatGPT. Same text box, same conversational flow, same satisfying rhythm of typing a question and watching an answer materialize. The first question I asked myself was simple: *What kind of project can I tackle with Claude?*

The answer surprised me. Because it wasn't code.

The Strategic Awakening

My very first substantive conversation with Claude wasn't about building something — it was about *thinking* about something.

I had been drafting a memo to our Director of Product Marketing. The core thesis was about a strategic tension I'd been mulling over for weeks: the difference between being best-in-class at coding agents and being best-in-class at building tools that coding agents call. For a company like MathWorks, this wasn't an abstract distinction — it was the question that would shape our positioning in the age of agentic AI.

I won't share the full memo here, but I can tell you what happened when I brought it to Claude. I expected what I'd gotten from ChatGPT — a polished summary, maybe some rephrased bullet points, the kind of agreeable helpfulness that makes you feel good but doesn't make you think harder.

Instead, Claude fact-checked my market claims. It confirmed that Cursor had crossed $1B ARR, that Claude Code had reached a $1B run-rate within six months of its launch. Then it stress-tested my arguments — praised the strategic logic but called out gaps. Where were the concrete examples? What about the risks I'd glossed over? The success metrics were too vague. The feedback was direct, specific, and genuinely useful.

I sat with that for a moment. This wasn't what I was used to from AI conversations. ChatGPT would have told me my memo was great and offered to make it longer. Claude told me where it was weak and how to make it stronger.

This set the tone for everything that followed: Claude as thinking partner, not just tool.

I went back and rewrote sections of the memo. The final version was meaningfully better — not because Claude wrote it for me, but because Claude pushed back on the parts that needed pushing. It was the kind of feedback you'd hope to get from a sharp colleague, delivered without ego or politics.

That experience stayed with me. Sitting with a sharper memo on one side of the conversation, I found myself thinking not just about the argument I'd improved, but about what the interaction itself *meant*. I had just been pushed to think harder — by a machine. Not nudged toward an answer, not flattered into confidence. Actually challenged. I wondered how many other people were having this moment, and how many still weren't.

I was also creating a presentation on the impact of agentic AI on the market of scientific and technical computing — something that most companies must have been asking themselves at that point. One diagram I developed described the chasm between the early adopters of agentic AI, evaluating solutions like Claude, and the majority still associating AI with chatbots like ChatGPT. That chasm was real, and I was watching myself cross it in real time.

The Blog Posts Begin

Within the first week, the strategic conversations spilled over into something more tangible. I had taken over the MATLAB AI Blog on September 1st, and now, with Claude as a collaborator, I found myself drafting posts at a pace I hadn't thought possible.

The list grew quickly:

- MATLAB MCP Core Server — the foundational post about connecting MATLAB to the agentic ecosystem
- Connecting to Hardware — data acquisition and image acquisition devices via MCP
- Deploying a Quant Agent — financial analysis with MATLAB as the computational backbone
- Antigravity + MATLAB — a playful post that turned out to be wicked good
- Giving Local AI Agents the Ability to Use MATLAB with MCP
- Building an Optimization Agent
- Automating the Documentation of MATLAB Analysis

A series of posts in the pipeline within the first week. This wasn't because I suddenly became a faster writer. The content was still mine — the ideas, the technical depth, the perspective on where MATLAB fit in the agentic AI landscape. What changed was the workflow.

Claude handled the boring parts. The formatting, the structuring, the initial research and benchmarking, the first-pass drafts of sections where I knew what I wanted to say but didn't want to stare at a blank page. This freed me to concentrate on the two things that actually mattered: **ideation and code.** The thinking and

the building. Everything else was scaffolding, and Claude was excellent at scaffolding.

Claude Analyzes My Voice

Then something happened that I hadn't expected.

I asked Claude to study my published writing — the blog posts, the technical articles, the presentation scripts — and tell me what patterns it found. I wanted to know if my writing had a recognizable voice, and if so, whether Claude could capture it.

What came back was a plain text file. A `.md` file, formatted like any other Markdown document, that Claude called a "skill." Not code. Not a configuration. Just structured prose describing how I write — and instructions for how to apply it.

```
---
name: yann-blog-voice
description: Write in Yann Debray's esta
blished blogging style for MathWorks tec
hnical posts
---

## Voice & Tone
Conversational but technically grounded.
 Acknowledge what you don't know.
Make the reader feel like a peer discove
ring something alongside you.

## Signature patterns
- **Opening hooks**: "If you are like me
, you are probably familiar with..."
- **Colleague references**: "my dear col
```

```
league [Name]"
- **Honest limitations**: "I'm not an ex
pert in Finance!"
- **Closings**: "Happy hacking, and let
me know what you build! 👋"
- **Structure**: Problem → Demo → Walkth
rough → Resources
```

I read through it with a strange mix of recognition and surprise. These were patterns I'd developed over years of writing, and I'd never consciously catalogued them. Claude had done it in a single conversation.

More importantly, it *worked*. When I asked Claude to draft sections using this skill, the output sounded like me. Not a generic approximation of a tech blogger, but specifically like me — my rhythms, my tendency to acknowledge what I don't know, my habit of opening with a shared experience. It was the difference between a cover band playing the notes and actually capturing the feel.

The writing voice skill was just the first one. I quickly saw that the format could capture almost any repeatable workflow. Converting conference notes into an internal presentation. Turning a YouTube transcript into a structured blog post. Taking a MATLAB analysis and automatically generating its live script documentation — something I later wrote about in detail on the MathWorks blog. Each of these was a transformation: input of one type, output of another, with the skill defining the rules of conversion.

The more ambitious examples came later: a skill for embedded AI workflows, walking Claude through the steps of taking a machine learning or deep learning

algorithm and targeting it to a chip. The kind of pipeline that normally lives in an engineer's head — implicit, tribal, undocumented. A skill made it explicit.

But I want to be honest about the limitation. Skills are non-deterministic. They're just text. Claude reads them and interprets them, and that interpretation can vary. Two runs of the same skill won't produce identical results. This isn't a bug — it's the nature of language models — but it's a real constraint if you need reproducibility.

What made people love them anyway was the simplicity. No code to write. No system to configure. Just a Markdown file that says: *here's how this works, here's what good looks like, do it this way.* My colleagues started gravitating toward skills almost immediately. They recognized them as something they'd always needed but never had a format for: a way to capture their own know-how and make it portable.

"I can write down how I do competitive analysis," one colleague told me. "And then Claude can do it the same way."

That's the idea. Skills as institutional memory. Skills as the interface between human expertise and AI execution.

There's a more dramatic way to put it — involving The Matrix and Keanu Reeves downloading kung fu in seconds — but that story belongs to Chapter 7, when we actually watched it happen live.

This was when I started to understand that what was happening went beyond productivity. A tool that

makes you faster is useful. A tool that learns your voice and helps you sound more like yourself — that's something else entirely.

Something Was Shifting

By the end of that first week with Claude, I had:

- Sharpened a strategic memo through genuine intellectual sparring
- Drafted the foundations for several blog posts
- Had my writing voice analyzed and codified in a way I could actually use

But the numbers don't capture what was really changing. It was the *quality* of the interaction. Each conversation with Claude built on the ones before it. There was continuity — a shared context that grew with every exchange. When I mentioned the "agent calling toolboxes" strategy, Claude knew what I meant. When I referenced the blog voice, Claude could apply it. When I circled back to an idea from two days ago, I didn't have to re-explain everything from scratch.

The conversations had something I'd never experienced with an AI before: **honest feedback.** Claude didn't just agree with me. It pushed back. It identified weaknesses. It suggested alternatives I hadn't considered. And it did all of this without the social dynamics that make human feedback complicated — no ego, no office politics, no tiptoeing around seniority.

What had started as a productivity experiment — can this tool help me write blog posts faster? — was becoming something I didn't have a name for yet. It wasn't a chatbot. It wasn't an assistant. It was starting to feel like a colleague who happened to be available at 10pm and never needed coffee.

I didn't know where this was headed. But I knew that the one-shot, disposable AI interactions of the ChatGPT era were behind me. Whatever came next would be built on continuity, shared context, and the kind of thinking partnership I was only beginning to understand.

Looking back, I'd later distill this first week into a slide I called "Agentic 101" — the foundation layer. Claude Desktop as the gateway drug. Python first, then MATLAB. It was the simplest of the four weekly lessons I'd eventually present to colleagues, but it was also the most important. Everything that followed — the philosopher tools, the agent orchestration, the terminal becoming home — grew from this first week of unlearning what I thought AI could do.

The philosopher tools were about to arrive. But that's the next chapter.

CHAPTER 3: THE PHILOSOPHER TOOLS

The philosopher tools were about to arrive. That's how I closed the last chapter, and I meant it literally.

Week 2 brought a shift I hadn't anticipated. Claude wasn't just a thinking partner anymore — it was a building partner. The strategic memos and blog drafts from Week 1 had been intellectually rewarding, but they were still fundamentally about *writing*. Now I found myself doing something different: building software. Real, distributable, reusable software. And the tools that emerged came with names I didn't expect.

FROM SKILLS TO TOOLS

The key insight of Week 2 was deceptively simple: **density — less is more.** Build tools, not just skills.

Let me explain. In Week 1, Claude had helped me create "skills" — structured prompts that captured my writing voice, my presentation style, my analytical frameworks. They were useful, but they were also fragile. A skill lives in a conversation. You invoke it, it shapes the output, and then it vanishes when the session ends. It's a one-shot thing — powerful in the moment, but not reusable in any systematic way.

A Command Line Interface tool is different. A CLI tool is a piece of software that lives on your machine, that

agents can call deterministically, that follows a workflow you've defined and tested. It doesn't depend on the quality of your prompt or the mood of the conversation. It just runs.

This distinction — between a prompt that works once and a tool that works every time — turned out to be the difference between playing with AI and actually building with it. And once I understood that, the tools started coming fast.

The Army of Philosophers

If you are like me BCC, you probably name your projects something boring and descriptive — `video-compress`, `slide-generator`, `media-converter`, `search-tool`. My first python tool was called `yt-transcripts`. Functional. Forgettable. I'd done this a lot. Before Claude.

But something about building with Claude changed the naming instinct. Maybe it was the late-night energy. Maybe it was the reading I'd been doing all year. Maybe it was the feeling that these tools deserved names that carried meaning beyond their function. Whatever the reason, I started naming my tools after philosophers.

And once I started, I couldn't stop.

montaigne.cc was the first. A python tool for presentation animation — transcription, translation, audio and video processing, all powered by Google Gemini AI. Montaigne's *Essais* are the original explorations — a mind wandering across languages, cultures, and ideas, interpreting everything it

encounters through the lens of personal experience. The tool does the same thing, in its own way. It takes content in one language and interprets it into another. It listens to audio and renders it as text. It crosses boundaries between media the way Montaigne crossed boundaries between disciplines.

nietzsche.cc came next. A presentation generator — a CLI where the `power` command generates slide decks. "The will to present." I know, I know. But hear me out. Nietzsche's Übermensch is about self-overcoming, about transcending the tedious and mundane to reach something higher. If you've ever spent three hours manually formatting a PowerPoint deck when the ideas were already clear in your head, you understand the appeal. The tool was my Übermensch moment — a way to transcend the drudgery and focus on what mattered: the ideas themselves.

There's another Nietzschean concept that fits even better: eternal recurrence. The idea that you should live as though you'd have to repeat every action for eternity. In automation, this is exactly what you want — a workflow so clean, so reliable, that running it again and again produces consistent results. Create, iterate, reuse. Eternal recurrence, but for slide decks.

ricoeur.cc was the one I was most proud of conceptually. A CLI search tool for conversation history — a way to dig through past dialogues and extract meaning from them. Paul Ricoeur's hermeneutic circle is the idea that you understand the parts through the whole, and the whole through the parts. Part, whole, part, whole — an endless loop of interpretation and reinterpretation. That's exactly what searching through conversation history feels like. You find a

fragment, it reframes your understanding of the entire exchange, and that new understanding sends you looking for different fragments. The tool and the philosophy were almost uncomfortably aligned.

I even asked Claude to evaluate the naming. The feedback was characteristically direct: ricoeur.cc edged ahead on conceptual coherence — the mapping between the tool's purpose and the philosopher's theme was the tightest. But nietzsche.cc was more polished in UX, more immediately usable. Claude recommended building a shared landing page to tell the story of why these tools existed as a family. Not just a collection of utilities, but a suite with a philosophical backbone.

An army of philosophers. That's what I started calling them.

Why Philosophers?

The naming wasn't random. It grew out of a reading list that had been building all year — a personal arc through books that, looking back, were preparing me for exactly this moment.

It started with Kafka's *The Trial*. The individual against the system, navigating opaque bureaucracies that operate by rules no one fully understands. If you've ever tried to introduce AI tooling into a large organization, you know this story intimately. The committees, the approval processes, the security reviews — Kafka wrote the manual on corporate AI adoption before AI existed.

Then Camus's *The Stranger*. Absurdism. The idea that life doesn't owe you meaning, so you'd better create your own. There's something deeply Camusian about vibe coding — this acceptance that the process is absurd, that you're building software by describing it in plain language to a machine, and that the appropriate response is not to demand it make sense but to embrace the creative freedom it offers.

Nietzsche's *Thus Spoke Zarathustra* was the big one. Will to power, self-overcoming, the three metamorphoses. More on that in a moment.

Plato's *Apology* — Socrates on trial, defending the examined life. Intellectual humility as a practice. The idea that wisdom begins with knowing what you don't know. Every time I typed "I'm not an expert in Finance!" into a blog post, I was channeling Socrates without realizing it.

Irvin Yalom's *The Schopenhauer Cure* bridged philosophy and practice — the idea that thinking about the world and doing something in the world aren't separate activities. Philosophy as therapy. Code as philosophy.

Then the modern voices: Harari's *Nexus*, framing AI as the next chapter in the human information story. Mollick's *Co-Intelligence*, providing the practical frameworks for working alongside AI systems.

"The existentialists ask 'how should I live authentically?' while Mollick and Harari ask 'how should we live with AI?' These questions are not separate. The tools we build shape the lives we lead."

That synthesis emerged during a late-night conversation with Claude, and it became the philosophical thesis underneath everything I was building. The tools weren't just utilities. They were answers — my answers — to questions that philosophers had been asking for centuries.

The Zarathustra Project

The philosopher tools and the reading list converged most dramatically in the Zarathustra project. I built a full voiceover script for a fifteen-slide presentation on *Thus Spoke Zarathustra*, and I built it using nietzsche.cc itself. The tool and its namesake, working together.

The presentation traced the arc of Nietzsche's masterwork: the prophet Zarathustra descending from his mountain after ten years of solitude. The Übermensch — not a superhero, but a concept of human self-overcoming. The three metamorphoses: the Camel who bears the weight of tradition, the Lion who rebels and creates freedom, the Child who plays and creates new values. The Last Man — Nietzsche's warning about a humanity so comfortable it stops striving. The Will to Power as creative drive, not domination. Eternal Recurrence as the ultimate test of how you choose to live.

Twelve to fifteen minutes of narrated content, generated from a tool I'd built with an AI, about a philosopher whose ideas had inspired the tool's name. If Nietzsche had known about recursion, he would have appreciated the approach.

The Philosopher Rap Battle

And then — because creative momentum has a logic of its own — the rap battle happened.

Once you've named your tools after philosophers and built a presentation about Zarathustra, the next step is obviously an educational web app where philosophers debate each other in rap format. Obviously.

The concept was simple: pick two philosophers, set them against each other on a topic, and let the arguments fly — but in verse. Nietzsche vs. Schopenhauer was the flagship matchup. Master and student. Pessimism vs. affirmation. "The world is suffering" vs. "Embrace it all and will it again."

It was playful. It was absurd. It was exactly the kind of project that happens when you're building with a collaborator who doesn't get tired, doesn't judge your ideas, and responds to "What if we made philosophers rap-battle each other?" with genuine creative energy rather than a raised eyebrow.

This is something I didn't expect about building with Claude: the creative compound effect. Each project made the next one easier to imagine. The philosopher naming led to the reading list connections, which led to the Zarathustra presentation, which led to the rap battle. One idea feeding the next, accelerating, branching. Not because Claude was generating ideas autonomously, but because having a tireless collaborator meant I never had to stop and wonder "Is this too weird? Is this worth the time?" The answer was always: let's find out.

The Developer Workflow

Behind the philosophical naming and the creative experiments, Week 2 was also when I became a real developer. Or at least, a closer approximation of one.

Python for CLI is fire. I'd known Python before, but building command-line tools with it — using `click` and `typer` — felt different. These weren't scripts. They were software.

`uv` is life. The Python package manager that made everything faster — environment creation, dependency resolution, the whole toolchain. Once you've used `uv`, going back to `pip` feels like going back to dial-up.

Publishing to PyPI was the moment it felt real. My tools weren't just local scripts anymore. Anyone could `pip install` them. They existed in the world, with version numbers and documentation and download counts. Let the game begin.

GitHub Issues became my requirements management system. Instead of keeping a mental list of features to add, I'd have Claude generate issues — structured, labeled, prioritized. Then I'd work through them methodically. Requirements in, planning, code out, tests validating, CI/CD deploying.

Speaking of tests: automated testing via GitHub Actions meant that every push was validated. And when a test failed, I could ask the GitHub MCP server to retrieve the trace. No more copy-pasting error logs from one window to another. The feedback loop tightened until it was nearly instantaneous.

I even recorded a YouTube demo — the whole workflow from concept to published package. Because

if you build an army of philosophers and don't document it, did it really happen? (like the saying goes, "if a tree falls in the forest and no one is around to hear it, does it make a sound?")

Tools as Self-Portraits

Here's what I think Chapter 3 is really about.

Naming your tools after philosophers isn't vanity — it's a way of encoding your values into the things you build. Every developer names their projects. Most names are functional, forgettable, interchangeable. But when you name a tool after Ricoeur, you're making a claim about what the tool means, not just what it does. You're saying: this search tool embodies a philosophy of interpretation. This presentation generator carries a vision of self-overcoming. This media toolkit is an act of translation in the deepest sense.

The philosopher tools were the first sign that my collaboration with Claude was producing something more personal than productivity gains. The blog posts from Week 1 were interesting. The strategic memos were valuable. But the tools — the tools were self-portraits in code. They reflected not just what I could build, but how I thought about the world.

Week 2 had taught me that the line between philosophy and programming was thinner than I'd imagined. The existentialists and the engineers were asking the same question: *What should I build, and why does it matter?*

What I didn't know yet was that these weekly discoveries — the density principle, the philosopher

army, the shift from skills to tools — would become the backbone of an internal evangelism campaign. Slides I'd present to three different audiences over the next six weeks, each time with a deeper understanding of what "build tools, not skills" really meant. But that story belongs to a later chapter.

I didn't have a complete answer yet. But I had an army of philosophers, a PyPI account, and a growing suspicion that the most interesting work was still ahead.

The competitive intelligence reports were about to begin. Claude was about to become my research partner. But that's the next chapter.

CHAPTER 4: THE COMPETITIVE INTELLIGENCE MACHINE

The philosopher tools had given me a taste of building — and I'd discovered that I could build pretty much anything I could think of. Which meant it was time to think more carefully about what I should build.

With great power, as they say, comes great responsibility. Just because Claude could turn an idea into a working tool in an afternoon didn't mean every idea deserved that treatment. So my interactions with Claude started shifting. Before diving straight into the code, I began taking more time to plan: What problem am I actually solving? Who is this for? Does this need to exist?

That shift opened up a different kind of conversation entirely. Less building, more thinking. And it turned out Claude was just as useful here — maybe more so.

Claude was becoming my research partner. And the research was getting serious.

THE PATTERN

As a product manager at technology companies, I've spent a meaningful fraction of my career doing competitive intelligence. Reading analyst reports. Scanning press releases. Sitting in meetings where someone presents a slide that says "Market Size: $X

Billion" and everyone nods as if the number is more than an educated guess.

The traditional workflow is slow and fragmented. You gather information from a dozen sources, synthesize it in your head, write it up in a memo, circulate it for feedback, revise, and eventually arrive at something useful — weeks after you started. By then, the market has moved (especially right now if you're covering AI).

With Claude, a new pattern emerged. It was simple, repeatable, and remarkably effective:

1. **I bring the question.** A company, a technology, a market segment. Something I need to understand deeply for strategic planning. *ex: "Look at the space of machine learning platforms and MLOps".*

2. **Claude first asks for clarification. Then it researches broadly.** Over 500 sources, tens of searches. Not just surface-level facts — deep dives into market dynamics, competitive positioning, technology trends, customer segments.

3. **Together, we extract implications.** This is the part that matters. Not "what is the market size?" but "what does this mean for us? Where are the opportunities? Where are the threats?"

4. *(Optional)* **NotebookLM turns the research into a presentation.** Once Claude produces a comprehensive report, I'd feed it into Google's NotebookLM and ask it to generate a slide deck. Two tools, two different strengths: Claude for the

depth of research and strategic synthesis, NotebookLM for structuring the output into something you can actually present in a meeting.

This wasn't just fact-gathering. It was thinking partnership applied to strategy. And the volume of what we produced shocked me.

The PIDO Deep Dive

The first major competitive intelligence effort was about a market I'd been tracking for quite some time: Process Integration and Design Optimization — PIDO, for those who live in the engineering software world.

In a traditional workflow, building a comprehensive market analysis of PIDO would take me the better part of a week. I'd need to identify the key players, look up their websites, understand their positioning, map the competitive landscape, estimate market sizes from fragmented data sources, and connect it all to MathWorks' strategic positioning.

With Claude, I typed a single prompt: `Perform a deep technological and market analysis of the field of Process Integration and Design Optimization […]`

What came back was a comprehensive analysis: a $200–220M USD market in 2024, growing at 10–12% CAGR. But the numbers were the easy part. What made it valuable was the strategic layer — the implications for MathWorks' positioning, the gaps in our current approach, the segments where we were strong and the ones where we were vulnerable.

I sat with the output for a while, cross-checking claims, testing the logic. Some of the analysis I'd adjust. Some of the market estimates I'd refine with internal data Claude didn't have access to. But the foundation was solid. The synthesis was genuinely useful. And it had taken an hour instead of a week.

This is the part that most people misunderstand about AI-assisted research. They imagine you press a button and receive a report. That's not how it works — or at least, that's not how it works well. What actually happens is a dialogue. I'd push back on Claude's framing. Claude would defend it or adjust. I'd bring internal context — things I knew about our product roadmap, our customer conversations, our engineering investments — that Claude couldn't access. Claude would integrate that context and sharpen the analysis.

The output was neither mine nor Claude's. It was ours.

The Neuroscience Toolbox Analysis

The second example was more technical and, in some ways, more revealing. Industry marketing needed a competitive analysis of the neuroscience research tools ecosystem — specifically, how MATLAB-based tools compared to the rising Python alternatives.

Claude dove into the landscape: FieldTrip, EEGLAB, MNE-Python, and the constellation of smaller tools orbiting them. The research surfaced something I hadn't fully appreciated. EEGLAB was leading in academic citations at 21,700+. But MNE-Python was showing the fastest growth trajectory — 3,200 GitHub stars, 416 contributors, and the kind of community momentum that's hard to reverse.

The strategic insight was pointed: a clear bifurcation was emerging between MATLAB-based tools (historically dominant, deeply embedded in research workflows) and Python-based alternatives (rapidly growing, zero licensing costs, open-source community energy). This wasn't just a technical comparison. It was a strategic signal about where the neuroscience research community was heading.

This is the kind of analysis that, delivered in a deck to industry marketing, changes how people think about their market. It emerged from a conversation, not a consulting engagement.

MATLAB Community Intelligence

The third stream of research was the most internally focused. Colleagues had started coming to me with requests.

"Can you tell me what users say about the Live Editor?" — that was Rohan.

"Can you tell me if a feature like Live Editor is useful and loved?" — that was Eric.

These weren't idle questions. They were the kind of product intelligence that normally requires weeks of manual analysis — combing through MATLAB Answers, reading forum threads, categorizing sentiment, identifying patterns. The kind of work that everyone agrees is important and nobody has time to do.

Claude changed the equation. By analyzing MATLAB community data — forum posts, questions, answers, the whole corpus of user-generated content — we

could extract product intelligence at a pace that made it practical. Not perfect. Not a replacement for proper user research. But a first pass that was fast enough to be useful and comprehensive enough to be informative.

The fact that colleagues were bringing me their questions told me something important. The competitive intelligence work wasn't just useful to me — it was becoming useful to the people around me. The research partner I'd found in Claude was turning into something closer to a research service.

Conference Intelligence

A fourth stream emerged from a different direction: conferences.

We had gone to JupyterCon in San Diego with my colleague Rohan in early November — before Claude, before the pilot. Delivering the internal report-out afterward had taken us the better part of a week. Distilling two days of talks and hallway conversations into something useful for colleagues who hadn't been there is harder than it sounds.

I wondered how much of that could be automated. When PyData came to Boston — its first time there, usually it's in NYC — I tested a new workflow: raw notes in, structured presentation out, with Claude handling the transformation in between. The quality was surprisingly good, and the time savings were real.

Conference intelligence follows the same pattern as competitive intelligence: breadth of input, synthesis under time pressure, delivered to people who need the

signal without the noise. Claude was well-suited to all three.

What Claude Brought, What I Brought

Here's the honest accounting of who contributed what.

Claude brought breadth. The ability to survey a landscape quickly, to pull together information from diverse sources, to identify patterns across large bodies of data. When I asked about the PIDO market, Claude could synthesize information from analyst reports, company announcements, technology trends, and market dynamics — the kind of breadth that would take me days to assemble manually (and Claude can hold way more information in his "head").

I brought depth and judgment. The ability to say "that market estimate feels high" or "you're missing the political dimension of this competitive dynamic" or "that's technically correct but strategically irrelevant." I knew which of our products competed where. I knew which customer segments we were winning and which we were losing. I knew the internal politics — which initiatives had executive sponsorship and which were struggling for resources.

Together, we produced synthesis. Neither of us could have done it alone. Claude without my judgment would produce technically accurate but strategically naive reports. My judgment without Claude's breadth would produce insightful but narrow analyses that took too long to assemble. And I was missing crucial parts of the

puzzle, blind spots that AI could fill in. The combination was more than the sum of its parts.

This is the pattern that Ethan Mollick describes in *Co-Intelligence* — the idea that human-AI collaboration isn't about automation but about augmentation. You don't replace the human. You amplify them. And you don't use AI as a search engine. You use it as a thinking partner.

The Sleepless Nights

There's a moment that Mollick writes about — the first time AI does something that makes you lie awake at night. Not in fear, exactly, but in a disorienting sense that the ground has shifted.

My colleague Kyle Hakes had his moment during the marketing book club I organized around *Co-Intelligence*. He told the group that someone had replicated six months of his market analysis work with a single deep research prompt — in thirty minutes. "70% of the way there," he said. His reaction: "I was like, well, I'm out of a job."

He was joking. Mostly. But the feeling was real. When you've spent years developing expertise in market analysis and competitive intelligence, and then you watch an AI produce something that's 70% as good in 0.1% of the time, the natural response is existential unease.

I felt it too. The competitive intelligence work I was doing with Claude wasn't just faster than my old workflow — it was a different thing entirely. The old workflow was labor-intensive and slow, which meant I

was selective about what I researched. I only deep-dived into markets that justified the time investment. With Claude, the marginal cost of research dropped close to zero. I could explore markets I'd never have bothered with before, just because the friction was gone.

This changes the job. Not eliminates it — changes it. The product manager who spends 80% of their time gathering information and 20% thinking about it now spends 20% gathering and 80% thinking. The bottleneck shifts from research to judgment. And judgment, it turns out, is the part that's hard to automate.

The Quality Question

I want to be honest about something. Not everything Claude produced was gold.

Some market estimates were off — based on outdated data or extrapolations that didn't account for recent disruptions. Some competitive analyses missed nuances that only someone embedded in the industry would catch. Some strategic recommendations were plausible but generic — the kind of thing any competent consultant would suggest.

The 70% number Kyle mentioned is about right. On a first pass, Claude gets you 70% of the way to a solid competitive intelligence report. The remaining 30% — the part that requires insider knowledge, strategic nuance, and the kind of judgment that comes from years of sitting in customer meetings — that's still you.

But here's the thing: 70% in thirty minutes versus 100% in two weeks is not a close comparison. The first pass gives you a foundation to build on, a framework to critique, a starting point that's dramatically better than a blank page. And the iteration from 70% to 95% — adding your judgment, your context, your strategic perspective — takes hours, not weeks.

The competitive intelligence machine wasn't perfect. But it was fast, it was comprehensive, and it freed me to do the part of my job that actually requires a human: deciding what the information means, what to include and what to leave out.

What This Changed

By the end of those early weeks, I had produced competitive intelligence reports covering markets I'd never have had time to analyze before. Each one followed the same pattern — Claude's breadth plus my judgment, iterated through conversation until we arrived at something genuinely useful.

But the bigger change was more subtle. My relationship to information itself had shifted. I'd gone from being a person who carefully selected which questions to research (because research was expensive in time) to a person who could research anything, anytime, and focus my energy on the higher-order question: *So what?*

The "so what" is where product managers earn their keep. And Claude had cleared the path to get there faster.

The competitive intelligence work was happening alongside the philosopher tools, alongside the blog posts, alongside the strategic memos. All of it feeding into a growing body of work that would need a unifying infrastructure. That infrastructure had a name — the Model Context Protocol — and it was about to change everything. But that's the next chapter.

CHAPTER 5: THE MCP STANDARDIZATION

Everything I'd built so far — the philosopher tools, the competitive intelligence reports, the blog posts, the strategic memos — had one thing in common. They were separate. Each tool lived in its own world. Each workflow started from scratch. The blog posts didn't know about the CLI tools. The research didn't feed back into the presentations. The competitive intelligence existed in documents, not in systems.

What I needed was infrastructure. A way to connect all of these pieces so that agents could use them together, not just one at a time. That infrastructure had a name, and while it didn't change everything on its own, it compounded with everything else I was learning.

The Model Context Protocol. MCP.

USB-C FOR AI

If you are like me, you remember the era of proprietary chargers. Every device had its own connector — a different shape, a different voltage, a different cable cluttering your desk drawer. Then USB-C arrived and suddenly everything plugged into everything. One standard. Universal compatibility. The end of cable chaos.

MCP is USB-C for AI.

That's the metaphor that stuck. Not because I coined it — it was already circulating in the developer community — but because it captured exactly what was happening. Before MCP, connecting an AI agent to an external tool meant building a custom integration for every combination. Claude needs to use MATLAB? Build a bespoke bridge. Copilot needs to access your database? Write another one. Every agent-tool pair required its own plumbing.

MCP changed the equation. It defined a standard protocol — a universal connector — that any AI agent could use to talk to any tool. Build one MCP server for your tool, and suddenly every MCP-compatible agent can use it. Claude Code, GitHub Copilot, Cursor, Windsurf — all of them, through the same interface.

For someone working at MathWorks, this wasn't an abstract standard. It was a strategic imperative.

THE MATLAB MCP CORE SERVER

The MATLAB MCP Core Server became the centerpiece of my work. I wrote about it, created tutorials for it, built onboarding materials around it, and positioned it as the bridge between MATLAB and the agentic AI ecosystem.

The server itself was elegant in its simplicity. Five tools:

1. **`detect_matlab_toolboxes`** — Reports which toolboxes are installed. Simple, but essential. An agent needs to know what capabilities are available before it can plan how to solve a problem.

2. **`check_matlab_code`** — Runs MATLAB's built-in `checkcode` function for style and correctness analysis. The agent can lint your code before running it.

3. **`evaluate_matlab_code`** — Executes a string of MATLAB code and returns the results. This is the core — the ability for an agent to actually *use* MATLAB, not just talk about it.

4. **`run_matlab_file`** — Executes a `.m` script file and returns the output. Where `evaluate_matlab_code` runs a snippet, this runs a full file.

5. **`run_matlab_test_file`** — Executes a MATLAB unit test file and returns comprehensive test results. The agent can not only write code but verify it.

Five tools. That's it. But those five tools turned MATLAB from a standalone application into something that any AI agent could call programmatically. The strategic implications were enormous.

I remember the moment it clicked for me — watching Claude Code call `evaluate_matlab_code`, run a computation, inspect the output, decide it needed to adjust a parameter, and call it again. No human in the loop. The agent was *using* MATLAB the way a user would. Not generating MATLAB code and hoping you'd copy-paste it (the ChatGPT pattern), but actually running it, reading the results, and iterating.

This is the difference between AI that generates code and AI that uses tools. MCP made the latter possible.

The Integration Play

Here's the positioning insight that emerged, and it was one of the most important strategic ideas I contributed during the pilot:

This is an **integration play**, not putting agentic front and center. MATLAB "integrates" with coding agents like Claude Code and GitHub Copilot. We're showing MATLAB users that they're not left behind.

The temptation for any established software company watching the agentic AI wave is to panic and try to build their own AI agent from scratch. "We need our own copilot! Our own agentic platform!" That's the instinct, and it's usually wrong.

The smarter play — the one I kept advocating — was integration. MATLAB doesn't need to be an AI agent. MATLAB needs to be the tool that AI agents reach for when they need to do serious computation. The MCP server was the bridge that made this possible.

I developed this into a pocket guide strategy aimed at two audiences:

Primary audience: MATLAB users evaluating coding agents. The people asking, "Does MATLAB work with these new coding agents I keep hearing about?" These are the users who've seen colleagues using Copilot or Claude Code and are wondering if they're being left

behind. The answer: no. MATLAB works with all of them, through MCP.

Secondary audience: MATLAB users who already use GitHub Copilot and want to extend it. They've got the coding agent — now they want it to actually run MATLAB, not just generate MATLAB code. The MCP server closes that gap.

"If you can think of it as an AND — if we can do for humans AND for AI, then maybe we have something here."

That became my closing line in more than one presentation. MathWorks serving both humans and AI agents. Not zero-sum. An AND.

The MCP Server Experiments

The MATLAB MCP Core Server was just the beginning. Once you understand the MCP pattern, you start seeing servers everywhere.

ThingSpeak — IoT data access via MCP. ThingSpeak is MathWorks' platform for IoT data storage and analysis. Wrapping it in an MCP server meant that any AI agent could query IoT sensor data, analyze trends, and trigger actions — all through the same standard protocol.

Google Spreadsheet — Private data access. This one started as a proof of concept and actually got deployed. My colleague and I built an MCP server that let agents read from and write to Google Sheets — a bridge between the structured world of spreadsheets and the conversational world of AI agents. It sounds mundane, but think about how many business workflows live in

spreadsheets. Giving agents access to that data, through a standard interface, unlocked an enormous amount of practical utility.

MATLAB Remote Service — For remote execution in the cloud or on a local server. This was the ambitious one. Instead of requiring a local MATLAB installation, the remote server let agents call MATLAB running somewhere else — in the cloud, on a compute cluster, on a colleague's machine. The MCP server abstracted away the location, so the agent didn't care whether MATLAB was local or remote.

Each server was a small project, but together they painted a picture: MCP wasn't just a way to connect one tool to one agent. It was the foundation for an ecosystem. Every server you build multiplies the capabilities of every agent that connects to it.

The Blog Post Machine

The blog posts I'd started drafting in Week 1 had a recurring character: MCP. Almost every post I published on the MATLAB AI Blog connected back to the protocol:

- **MATLAB MCP Core Server** — the foundational post, announcing the server and explaining the architecture

- **Connecting to Hardware** — data acquisition and image acquisition devices via MCP. The physical world, accessible to agents

- **Deploying a Quant Agent** — financial analysis with MATLAB as the computational backbone, connected through MCP

- **Giving Local AI Agents the Ability to Use MATLAB** — the on-ramp for users running local models

- **Building an Optimization Agent** — engineering optimization workflows, orchestrated through MCP

- **Automating the Documentation of MATLAB Analysis** — the full pipeline from analysis to documentation, with MCP to automate the boring stuff

Six posts, all variations on the same theme: MATLAB + MCP = MATLAB in the agentic era. The blog posts weren't just content marketing. They were proof of concept. Each one demonstrated a different use case, a different audience, a different reason why the MCP integration mattered.

The Onboarding Campaign

As I went deeper into the MCP ecosystem, I realized something: the technology was only half the challenge. The other half was getting people to use it.

So I built an onboarding package. A complete Claude Code 101 for MathWorkers: installation guide, CLAUDE.md files for common project types, custom skills for repeatable workflows, MCP server setup instructions, Git workflow best practices, tips and tricks I'd accumulated over weeks of daily use.

This is where the Claude Code features I'd been exploring became teachable material:

`/init` — Bootstrap a project with CLAUDE.md and initial configuration. For teams with an existing codebase, this was the fastest on-ramp: one command analyzes the project — its structure, its dependencies, its conventions — and generates a CLAUDE.md that makes every subsequent conversation more productive. I showed colleagues how `/init` on their MATLAB projects would immediately give Claude the context to be useful.

`CLAUDE.md` — The file that solved the "every session starts from zero" problem. If you've used ChatGPT, you know the frustration: every conversation begins with you re-explaining your project, your conventions, your preferences. CLAUDE.md is a Markdown file that Claude reads at the start of every session — like onboarding documentation for a new teammate who happens to have perfect recall. Your build commands, your coding standards, your architecture decisions — all persistent, all version-controlled alongside your code.

Custom skills — Chapter 2 introduced the concept: a plain-text `SKILL.md` file that captures a repeatable workflow. But there, skills lived in Claude Desktop — pasted in at the start of a conversation, powerful but manual. Claude Code changes the equation.

Skills Get a Home

Remember the blog voice skill from Chapter 2? That `SKILL.md` had to be reloaded every new session. In Claude Code, it gets a permanent address:

`.claude/skills/blog-post/SKILL.md`. Now I could type `/blog-post "MATLAB + Hardware via MCP"` and Claude would draft a post in my voice — every time, without the preamble, without reloading anything.

But here's where it got more interesting. I noticed that sometimes Claude should load my voice *without me asking*. If I was editing a blog draft and Claude suggested a paragraph, it should already sound like me — not because I typed `/blog-post`, but because Claude recognized the context and applied the right knowledge automatically.

That's what the description field in `SKILL.md` enables. Claude reads all skill descriptions at the start of every session. When it sees a match between the description and what you're doing — drafting a blog post, running a competitive analysis, generating a presentation — it loads the full skill on its own. You don't invoke it. It activates itself.

This was the progression I taught in the onboarding:

1. **CLAUDE.md** gives Claude persistent knowledge about your project

2. **Slash commands** (`/skill-name`) let you invoke a workflow explicitly

3. **Auto-invoked skills** let Claude recognize when a workflow applies and load it on its own

The difference between typing `/blog-post` and having Claude just *know* your voice is subtle but profound. It's the difference between a tool you pick

up and a capability that activates when the situation calls for it.

And once colleagues saw this, they started connecting it to everything they'd built. The philosopher tool documentation from Chapter 3? Skills. The competitive intelligence workflows from Chapter 4? Skills. The presentation generation, the conference report-outs, the embedded AI pipelines? All skills. Every repeatable workflow could now be formalized, version-controlled, shared with the team, and triggered automatically when the context was right.

Agents Calling Agents

And then things got weird. In a good way.

The MCP thesis was "agents calling tools" — an AI agent reaching out to external capabilities through a standard protocol. But somewhere around Week 3, a more advanced pattern emerged: **agents calling agents.**

Not just Claude using MATLAB. Claude orchestrating Codex and Gemini CLI. Multiple AI agents, working in parallel, each assigned a different piece of a larger task. Claude as the conductor, the other agents as the orchestra.

The inspiration came from a Youtuber called NetworkChuck — the idea of sending multiple agents down different research paths and synthesizing the results. Deep research, but parallelized. Instead of Claude doing everything itself, it could delegate: "You go research the market size. You go analyze the technical architecture. You go find the competitive

landscape." Then it would synthesize the results into something more comprehensive than any single agent could produce.

Gemini CLI's headless mode made this practical. OpenAI's Codex CLI offered another execution path. Even GitHub Copilot could be subcontracted through the `/github:AssignCodingAgent` MCP command — Claude assigning implementation work to Copilot through GitHub Issues.

This escalation from tool-use to agent-orchestration felt like watching a gear shift. The MCP server gave agents hands. Agents calling agents gave them a team. The metaphor wasn't a single worker with better tools anymore — it was a manager coordinating a team of specialists, each with their own strengths. Claude Code would later formalize this with *agent teams* — custom subagent definitions you could configure in `.claude/agents/`, each with its own model, tools, and instructions. The ad-hoc orchestration I was doing manually was becoming a first-class feature.

I showed this slide to the industry marketing team during my Week 3 presentation. I could see the moment it clicked for some of them — and the moment it overwhelmed others. The gap between "I use ChatGPT for emails" and "I orchestrate multiple AI agents in parallel" was wider than anyone had expected.

What MCP Actually Changed

Here's what I think the MCP shift is really about.

Before MCP, every AI integration was a one-off. You'd connect one agent to one tool, and the integration was fragile, proprietary, and hard to maintain. The cost of connecting anything to anything was high, so people connected very little.

After MCP, connection became cheap. Build one server, and every agent can use it. Build one client, and every server is accessible. The network effects kick in: every new server makes every agent more capable, and every new agent makes every server more valuable.

For MathWorks, this meant MATLAB could join the agentic ecosystem without rebuilding itself. The MCP server was the bridge — lightweight, standards-based, and infinitely extensible. For me personally, it meant the philosopher tools, the competitive intelligence workflows, the blog post pipelines, and the MATLAB integrations could all talk to each other through the same protocol.

The infrastructure I'd been looking for — the thing that would connect all the separate pieces of work into a coherent system — had been there all along, waiting for me to understand it.

And understanding it changed everything that came after. The side projects were about to get ambitious. But that's the next chapter.

CHAPTER 6: THE SIDE PROJECTS THAT KEPT GROWING

I'm not proud to admit it, but I have a graveyard of side projects. Half-finished repos, abandoned domain names, ideas that burned bright for an evening and cooled by morning. The normal lifecycle of a side project is: excitement, prototype, neglect, guilt, quiet deletion (if I ever come back to clean it up). Maybe you keep them around in a private GitHub folder as monuments to good intentions.

Something different happened this time.

It was early January, a few weeks into the new year. I was in the terminal one evening, working on something for the recipe site — I don't even remember what, probably a content pipeline — when I realized I had three different projects open in three different tabs, and all of them were *active*. Not in the "I should really get back to that" sense. Active as in: I'd made progress on each one in the past week, and each one had a clear next step waiting for me. I stopped typing and tried to remember the last time that had happened with a side project. I couldn't.

Over the previous chapters, I'd built an infrastructure I hadn't planned. The philosopher tools gave me reusable CLIs and a workflow for turning ideas into distributable software. The competitive intelligence work with Claude taught me how to research markets

rigorously instead of guessing. The MCP revolution connected my tools to each other and to external services in ways that multiplied what a single person could do. None of this had been designed as a system. It was more like — each piece created the conditions for the next piece to exist.

And then the side projects stopped being "side." They started growing on their own.

Not all of them. Not into businesses. But the pattern was unmistakable: projects that would normally have died after one session were instead evolving, branching, connecting to each other. The graveyard wasn't filling up anymore. Things were staying alive.

This chapter is about three of those projects — a recipe website that almost became a business, a YouTube channel that became a laboratory, and a music recommendation app that became a philosophical statement. Each one started as an island. Each one turned into something I didn't expect.

RECETTESDEZOE.COM — FROM DEMO TO BUSINESS?

You already know the origin story from Chapter 1. Late-night kitchen session, father-in-law watching over my shoulder, a recipe website conjured from a single ChatGPT prompt, domain purchased at 10pm. Then the surprise: 1,300 organic visitors four months later, with zero marketing, zero updates, zero effort.

What you don't know is what happened next.

The traffic numbers broke something in my mental model, I said that already. But breaking a mental model is one thing. Deciding what to build on the rubble is another. Because once I had evidence that people were actually visiting this site — cooking from it, maybe — I had to ask a question I hadn't planned on asking: *Is this a product?*

I did what any self-respecting product manager would do with an existential question about a 10pm side project. I reached for a framework.

The Mom Test

Rob Fitzpatrick's *The Mom Test* is the gold standard for product validation. The core idea is deceptively simple: stop asking people if your idea is good (they'll lie to be polite) and start asking about their actual behavior. Don't say "Would you use a recipe website?" Say "Tell me about the last time you looked for a recipe online. What happened?"

I couldn't interview the 1,300 visitors — I didn't know who they were. But I could do the next best thing: study the market they were already choosing from.

The Competitive Landscape

This is where the Chapter 4 pattern kicked in. I'd spent weeks training Claude to be a competitive intelligence partner — structured research, market sizing, strategic analysis. Now I turned that same lens on the French recipe market. And the landscape was sobering.

Marmiton — the absolute behemoth. 10 million monthly visitors, user-generated content, a brand so

embedded in French cooking culture that "je vais regarder sur Marmiton" is practically a reflex. **750g** (owned by Webedia) — professional content, video-forward, with the media infrastructure to dominate on search and social. **Cuisine AZ** — another giant, part of the same media group as 750g, covering every conceivable recipe with industrial-scale content production.

Then there was recettesdezoe.com. A site built in a kitchen. By a product manager. At 10pm.

The analysis was honest, and Claude didn't sugarcoat it: the French recipe market was a red ocean. The incumbents had decades of SEO authority, millions of indexed pages, and content pipelines that no solo operator could match on volume. The 1,300 visitors were real — probably finding the site through long-tail keywords that the giants hadn't bothered to optimize for. But scaling against Marmiton was like opening a corner bakery and hoping to outcompete an industrial bread factory.

The insight wasn't "give up." It was "compete differently." The big players wrote recipes like food blogs — SEO-optimized, template-driven, drowning in ads and pop-ups. recettesdezoe.com had something they didn't: a voice.

The Voice Test

This was the moment where Chapter 2's voice analysis came back in a new form. Remember the blogging workflow I'd developed with Claude — the careful attention to preserving an authentic writing voice, the resistance to letting AI flatten everything into generic

corporate prose? The same principle applied here, but the domain was entirely different.

I defined a quality test for any recipe the site would publish:

"Could this have been written by someone sharing their grandmother's recipe over coffee? If it sounds like a food blog template or a restaurant menu, it's wrong."

That was the moat. Not SEO. Not scale. Voice. The feeling that a real person had cooked this dish, tasted it, adjusted the seasoning, and wanted to tell you about it the way they'd tell a friend.

Content Scaling with Zoé

If voice was the moat, I still needed a way to produce content without spending every evening writing recipes by hand. This is where the Chapter 3 pattern kicked in again: build tools, not just skills.

I designed a CLI tool for recipe generation. I called it "Zoé" — the same name as my daughter, the same name that was already in the domain. If the philosopher tools got names that carried conceptual meaning, this one carried something more personal: a family name, a familial tradition. Where montaigne.cc and nietzsche.cc encoded philosophical values, Zoé encoded the domestic warmth that the whole project was supposed to embody.

The tool was built with the Copilot SDK and followed a structured workflow: research a recipe category, generate drafts that matched the established voice, validate against the Voice Test, and produce

Markdown files ready for the site. Claude was the engine, but the CLI was the guardrail — deterministic, repeatable, consistent. A skill would have drifted with every conversation. A tool kept the voice stable.

Did recettesdezoe.com become a business? No. Not yet, maybe not ever. But it stopped being a demo. It had a competitive analysis, a content strategy, a voice standard, and a production tool named after my daughter. The 10pm party trick had become something I could pick up, work on for an evening, put down, and come back to a week later without starting from zero.

That's what "kept growing" means. Not exponential growth. Not viral success. Just — alive.

The YouTube Channel That Became a Lab

@yannspeaksai started as a way to document my projects. Record what I'm building, explain the process, put it out there. The kind of thing a lot of technical people do — part portfolio, part accountability mechanism, part excuse to learn video editing.

Seven videos in 2025. Five different sports. Five completely different technology stacks. And without planning it, the channel became less of a content outlet and more of a laboratory — each video an experiment in a different domain, with the only constant being AI as a collaborator.

Tennis — Predicting the French Open

The first sports video was about prediction. I built an ELO rating system enhanced with machine learning to forecast French Open match outcomes. The model achieved 75% accuracy, which sounds impressive until you realize that tennis prediction is a well-studied problem and the best models in the literature hover around the same range. The point wasn't to beat the state of the art. The point was that with no background in sports statistics, I could build a credible prediction system in a few hours.

The stack was MATLAB for the analysis and Python for the data pipeline. Claude handled the boilerplate interop layer between the two — something that would have taken me days of Stack Overflow diving in before AI.

NBA — Dashboards and Automation

The basketball project pushed into web development territory. Season ranking animations, a React dashboard showing team statistics, and — the part I was proudest of — a full GitHub Actions CI/CD pipeline that automatically updated the data and rebuilt the visualizations.

This was the "build tools, not skills" philosophy applied to content creation. I didn't just make a dashboard. I built a system that kept the dashboard alive. Every new game night triggered an automatic update. The infrastructure outlasted the session that created it.

NFL — From Toolbox to Clutch Factor

The American football work was actually two projects, each built in a single day, a week apart, both vibe-coded with OpenAI Codex.

The first was the **NFL Analytics Toolbox** — a MATLAB-first interface to NFL statistics. The idea was to bridge MATLAB and the open-source `nfl_data_py` Python library, so you could pull weekly player stats, play-by-play data, and team rosters directly into MATLAB tables. I built high-level functions like `showTeamDashboard("KC", 2023)` that generated four-tile figures with yardage breakdowns, projected vs. actual wins, roster profiles, and top performer summaries. The whole thing was packaged as a proper MATLAB toolbox — with an `Open in MATLAB Online` badge, unit tests, and a setup script that auto-provisioned a Python virtual environment via `uv`. The first commit message tells the story: *"vibe coding v0 with Codex."*

A week later, I built the second project: the **TV Clutch Factor**. This one was pure Python into a web dashboard. The pipeline downloaded 25 seasons of NFL play-by-play data, filtered to clutch situations — score differential within 7.5 points in the final minutes of the fourth quarter or overtime — and computed an original metric that combined traditional passer rating with volume weighting. Which quarterbacks performed best when the game was on the line? Aaron Rodgers, Matthew Stafford, and Drew Brees topped the leaderboard. The results fed into an interactive HTML dashboard with a broadcast-style dark theme, sortable columns, and gold/silver/bronze podium styling, served directly from GitHub Pages.

Two projects, one day each. One MATLAB, one Python-to-web. Neither would have existed if the cost of building them had been measured in weeks instead of evenings.

Cycling — The Ghost That Kept Running

And then there was cycling. GhostRunners.app from Chapter 1 — the ride comparison tool built with MATLAB Mobile data capture. The clearest example of "kept growing" in the entire portfolio.

In the pre-Claude era, GhostRunners was a functional but rough prototype. MATLAB Mobile captured the data, MATLAB Online crunched the analysis, Python handled the Garmin FIT file parsing. It worked. I recorded YouTube videos about it. And then it sat there.

With Claude, I came back to it. The MCP server connected MATLAB to the broader workflow. The CLI patterns from the philosopher tools gave me a structure for the user interface. The CI/CD learnings from the NBA project meant I could automate the data pipeline. Each previous project had deposited a capability that GhostRunners could now draw on.

The ghost was still running. But now it was running faster.

Soccer — The Premier League Dashboard

The soccer project was the most MATLAB-native of all the sports experiments. Not a data pipeline feeding a web dashboard, not a Python/MATLAB hybrid — a pure MATLAB programmatic UI built as a single class file.

The idea started with a screenshot. I'd seen a Premier League standings dashboard somewhere — clean layout, color-coded zones, position-by-matchday chart — and thought: *I bet I can build that in MATLAB.* Not App Designer, not a MATLAB App with a `.mlapp` file that's impossible to diff or version-control. A programmatic `uifigure` with `uigridlayout`, written as a single `.m` class file. The kind of code that plays nicely with Git, that Claude can read and modify, that you can share with a link.

Two days. That's how long it took to go from an empty file to a polished dashboard with five iterations documented in screenshots — v0.1 through v0.5, each one closer to the target layout. The final version had a full standings table color-coded by Champions League spots (blue), Europa League (orange), and relegation zone (red). Click a team and the side panel showed their badge (fetched live from TheSportsDB API), their card counts, and a ten-match form guide rendered as colored blocks. A position-by-matchday chart tracked all twenty teams across the season, highlighting your selection in bold blue against gray background lines.

The part that surprised me was the test suite. I'd built `matlab.unittest` test cases with tagged categories — unit tests for the API name mappings, integration tests for the full badge-fetching workflow, performance tests verifying that cached lookups stayed under 10 milliseconds. This was a side project with better test coverage than some production code I'd seen. The pattern of building things *properly* — tests, documentation, clean architecture — had become a habit, not an overhead.

And the `Open in MATLAB Online` button in the README? That was a small detail that connected the project to my day job. Anyone could click that badge and run the dashboard in their browser, no installation required. A product manager dogfooding his own product, building the kind of showcase he'd want other users to create.

The Pattern

Here's what the YouTube channel taught me: **the marginal cost of exploration had dropped to near zero.**

Each video required learning a new domain — tennis statistics, NBA data APIs, NFL play-by-play formats, cycling biomechanics. In the old world, the domain learning alone would have consumed weeks. With Claude as a research and implementation partner, I could go from "I know nothing about ELO ratings" to "here's a working prediction system" in an evening or two.

Five sports. Five stacks. One year. The channel wasn't just documenting what I built. It was proving, video by video, that the barrier to entry for any technical domain had collapsed. Not to zero — you still needed taste, judgment, and the willingness to learn. But the activation energy had dropped from "this will take months, is it worth it?" to "let's find out this weekend."

Gaining Back Control Over the Algorithm

The music project started with an itch.

If you use Spotify — and statistically, you probably do — you've experienced this: the algorithm decides what you like. It feeds you recommendations based on your listening history, and over time, your taste and the algorithm's model of your taste begin to converge. You listen to what it suggests, which reinforces its model, which narrows its suggestions, which narrows what you listen to. The feedback loop is invisible and relentless.

I didn't hate Spotify's recommendations. Some of them were genuinely good. But I resented the passivity of it. My music taste was becoming a mirror of an algorithm's assumptions about me, and I wanted to break out of that loop.

The Architecture

The idea was simple in concept, ambitious in scope: build a personal music recommendation system using Spotify's own audio features — energy, acousticness, danceability, valence, tempo — but with *my* logic instead of *their* algorithm.

The core engine was a K-nearest neighbors (KNN) model. Take a song I loved, find the songs in my library that were most similar across those audio features, and surface the ones I'd forgotten about or never noticed. Not discovery of new music (Spotify already does that well) but rediscovery of music I already owned and had lost in the algorithmic shuffle.

Claude helped me design the system architecture and integrate the Spotify API. The data pipeline pulled my listening history and audio features for every track. The KNN model ran locally — no cloud dependency, no

algorithm I didn't control. The recommendations were mine, based on rules I could see and modify.

Form Factors

This is where the project got interesting. Once the recommendation engine existed, the question became: how do you interact with it?

I explored four different form factors, each one a different answer to that question:

Web app — the obvious first choice. A clean interface where you pick a seed song and see recommendations. Good for exploration, but it required a browser and a server.

CLI tool — because by this point, everything I built had a CLI. Type a song name, get recommendations in the terminal. Fast, scriptable, composable with other tools. The philosopher tool ethos applied to music.

Desktop app — for a more visual experience. Album art, audio previews, the kind of rich interface that a terminal can't provide.

Hybrid — the most ambitious version. A CLI that could launch a lightweight web interface when you wanted the visual experience, but stayed in the terminal when you wanted speed. The best of both worlds, if you could stomach the complexity.

Each form factor taught me something different about interface design. But the lesson that stuck was this: having the recommendation engine as a separate, well-defined module meant I could experiment with interfaces freely. The tool-first approach from Chapter

3 — building the engine before the interface — paid dividends I hadn't anticipated.

Use Cases That Multiplied

The recommendation engine was the seed, but the use cases kept branching:

Local music management — I had years of downloaded music scattered across hard drives. The same audio feature analysis that powered recommendations could also organize, tag, and deduplicate my local collection.

Personal Spotify Wrapped — Spotify's annual summary is fun but superficial. With raw access to my listening data and audio features, I could build something deeper: trends in my taste over time, the acoustic signature of my moods, the genres I gravitated to during specific seasons.

Mood-based playlists — Instead of Spotify's generic "Chill" or "Focus" categories, playlists generated from my own definition of what those moods sounded like, calibrated to my library, my features, my thresholds.

The Philosophical Resonance

Here's where the project connected back to something deeper.

In Chapter 3, the existentialist synthesis emerged: *"The existentialists ask 'how should I live authentically?' while Mollick and Harari ask 'how should we live with AI?' These questions are not separate."* The philosopher tools were answers to philosophical questions encoded in code.

The music project was the same impulse, just in a different domain. Building your own recommendation algorithm is an act of self-determination. It says: I refuse to let a platform decide what I like. I'll examine my own taste, build my own model of it, and use that model to rediscover what I've been missing. Socrates meets Spotify. The examined playlist is worth listening to.

"Gaining back control over the algorithm isn't about rejecting technology. It's about choosing which technology reflects who you are instead of who a platform thinks you should be."

This wasn't just a music project. It was the book's thesis — AI as amplifier of human intention — applied to the most personal domain I could think of: what sounds fill your house on a Sunday morning.

The Portfolio Effect

Let me step back and look at what had happened.

In Chapter 1, I described a pattern: *"Every conversation started from zero. Every project was an island."* recettesdezoe.com was a single-session creation. GhostRunners was a rough prototype that worked but couldn't grow. The YouTube channel was a series of disconnected experiments. Each project lived in its own bubble, with no shared infrastructure, no common patterns, no way for one to feed into another.

By the time I'd reached the projects in this chapter, that had changed entirely. The projects weren't islands anymore — they were nodes in a network.

recettesdezoe.com drew on the voice preservation techniques from Chapter 2 for its content standard, and on the "build tools, not skills" philosophy from Chapter 3 for the Zoé CLI and on the competitive intelligence patterns from Chapter 4 for its market analysis. The YouTube sports projects cross-pollinated: CI/CD from the NBA project fed into GhostRunners, MATLAB toolbox patterns from the NFL project informed the cycling analysis, and every project deposited knowledge that the next one could draw on. The music recommendation app used the same CLI architecture as the philosopher tools, the same research methodology as the competitive analysis, and connected back to the existentialist themes that ran through the whole journey.

The infrastructure chapters — the philosopher tools, the competitive intelligence machine, the MCP standard — hadn't just produced outputs. They had produced a *way of working*. A set of patterns and capabilities that made every subsequent project cheaper, faster, and more connected than it would have been in isolation.

This is the portfolio effect. Not a planned portfolio — I never sat down and said "I'm going to build an interconnected network of side projects." It emerged from the compounding of tools, patterns, and knowledge across dozens of conversations with Claude. Each project made the next one possible. Each tool reduced the cost of the next tool. Each experiment deposited a capability that future experiments could inherit.

I want to be honest about the limits. Most of these projects are still side projects. recettesdezoe.com isn't

generating revenue. The YouTube channel has a modest audience. The music app is something I use personally. None of this is a startup success story.

But that's precisely the point. The transformation wasn't about commercial outcomes — it was about the *cost of exploration*. When it takes weeks to build something, you have to be strategic about what you build. You plan, you evaluate, you calculate the expected return on investment. When it takes an evening, you can afford to follow curiosity wherever it leads.

"When exploration becomes cheap, you stop asking 'is this worth my time?' and start asking 'what am I curious about?'"

That shift — from strategic allocation of scarce building capacity to abundant, curiosity-driven exploration — is, I think, the most underappreciated consequence of working with an AI collaborator. The projects themselves matter less than the fact that they exist at all. Each one is evidence that the barrier between "I wonder if…" and "let's find out" has been nearly erased.

What the Side Projects Taught Me

The side projects taught me something that the work projects couldn't.

At MathWorks, the value of Claude was measurable — faster blog posts, sharper competitive analyses, better presentations. The ROI story was clear and presentable. But the side projects operated outside the

logic of ROI. Nobody asked me to build a recipe website or a music recommendation engine. Nobody was tracking the output. The only metric was whether I wanted to keep going.

And I kept going. That was the data point that mattered.

Because when you strip away the professional context — the reporting obligations, the pilot program requirements, the need to justify your tools budget — what you're left with is the raw question: *Do I want to spend my evenings doing this?* And the answer, project after project, evening after evening, was yes.

There's a specific kind of knowledge that comes from building things nobody asked you to build. Work projects teach you what's possible. Side projects teach you what's *exciting*. And when the time came to explain this journey to colleagues — to stand in front of a room and make the case that something fundamental had changed — I needed both kinds of knowledge. The work projects gave me the evidence: productivity gains, workflow improvements, measurable outputs. But the side projects gave me the conviction. The ability to say, with genuine enthusiasm and not just professional obligation, that this technology had changed how I spent my free time. That I *chose* to be in the terminal at 11pm on a Tuesday, not because anyone was watching, but because the next experiment was calling.

That conviction was about to matter more than I expected. Over the following weeks, I'd present to three different teams at MathWorks, each time refining the message, each time drawing on both the

professional evidence and the personal story. The internal evangelism campaign was about to begin — and with it, a new challenge: translating a felt experience into something that could change other people's minds.

But that's the next chapter.

Chapter 7: Writing the Future — Sharing Knowledge

By mid-January 2026, I had been using Claude Code for about six weeks. I'd built tools, sharpened strategies, drafted blog posts at a pace that surprised even me, and spent more late nights in the terminal than I probably should have. But now I faced a challenge that no amount of agentic wizardry could automate: explaining what I'd found to colleagues who hadn't experienced it.

The pilot had a simple requirement — report back on your usage in Q1. What started as a reporting obligation became something I hadn't anticipated: an internal evangelism campaign.

You might know the feeling if you are passionate about tech. You discover something that changes how you work, and suddenly you can't stop talking about it. The problem is that enthusiasm without structure is just noise. I needed to show, not tell. And I needed to tailor the message to audiences who cared about very different things.

The Presentation That Kept Evolving

The story of these presentations actually begins with that September 2025 deck Jacob and I built together

(remember chapter 1) — “GenAI Tools for Modern Tech Marketers.” Solid. Useful. Conventional.

By January, that deck had been transformed beyond recognition. Most of the original slides were gone. In their place was something called “Agentic Learnings” — and over the next six weeks, I would rewrite it every week, each time for a different audience. Each version wasn’t just a reshuffled slide order. Each one reflected genuine learning, additional weeks of experience, and a deeper understanding of what mattered to the people in the room.

Outbound Marketing — The Workflow Version

The first presentation landed on January 15th, Week 2 of the new year. I presented to the outbound marketing team, together with Eric, the head of product management for MATLAB. The subtitle said it plainly: “Learnings after a few weeks of Claude Code.”

I organized everything into three categories that would carry through all subsequent presentations:

Agentic MVP — building apps and prototypes. I posed the question the audience was probably thinking: “MVP for Minimal Viable Product or Minimal Viable Prototype?” The ambiguity was intentional. This wasn’t just Vibe coding web sites anymore. It was architecture work, guiding a senior developer (Claude).

Agentic Digital Marketing — automating the boring steps of the blogging workflow so you can concentrate on ideation and code writing. I showed the workflow diagram: Claude handles formatting, structuring, and

first-pass drafts. The human focuses on ideas and technical content.

Agentic Market Intelligence — this was the one that made people lean forward. I showed the PIDO example: Process Integration and Design Optimization market analysis from a single prompt. The output? A comprehensive analysis: "$200M USD market in 2024, growing at 10% CAGR," with strategic implications included. One prompt. What used to take a week of research distilled into minutes.

The outbound team needed concrete before-and-after comparisons of their daily work — blogging, competitive analysis, community monitoring. So that's what I gave them. This was the most workflow-oriented of the three decks.

Industry Marketing — The Technically Ambitious Version

A week later, same core material, completely different emphasis. The industry marketing team needed to understand the competitive landscape — that was their job, after all. So I added depth.

A new category appeared that hadn't existed in the outbound deck. **Agentic Development** showed Claude Code porting a Python package to MATLAB — a technical use case proving this wasn't just a marketing productivity tool.

Then came the research examples — and this is where the deck hit differently for this audience. The neuroscience toolbox analysis I'd run with Claude (covered in Chapter 4) wasn't just an interesting output

anymore. In this room, it was a mirror. These were the people responsible for deciding where MathWorks should invest in academic disciplines. Showing them that a bifurcation was already underway between MATLAB-based tools and Python alternatives wasn't a demonstration of what AI could do. It was a strategic conversation starter about their own market. The same analysis that had felt like a research experiment when I ran it became something closer to a briefing when I presented it to people with skin in the game.

The weekly learnings section had evolved too. By Week 3, the slides carried three layers of accumulating insight — from Agentic 101 through tool-building to multi-agent orchestration. The audience could see the learning curve accelerating in real time.

Educator Programs — The One Built from Scratch

By Week 6, I didn't just update the deck. I rebuilt it entirely.

The educator audience cared about teaching content creation, forum support, and MATLAB live script generation. The generic "blogging workflow" and "market intelligence" sections from earlier decks? Gone. Replaced with education-specific use cases: creating content for a linear algebra workshop at the University of Oklahoma, analyzing community toolboxes for academic adoption, generating teaching materials that could scale.

This was the most audience-adapted version. And it was the one where something unexpected happened.

The Learning Curve in Four Weeks

Each version of the deck included a “weekly learnings” section — a running log of what I’d discovered since the last presentation. What started as a simple bullet list became, without my planning it, a narrative of transformation. Four weeks, four stages.

Week 1: Agentic 101. Start with Claude Desktop. Get a taste of the agentic capabilities — the ability to read and write directly on the file system. Discover a few concepts: skills, tools, the Model Context Protocol. See how this applies to coding. Begin with Python, then connect to MATLAB via the MCP Core Server. This was the foundation — understanding that an AI agent isn’t just a chatbot with delusions of grandeur, but a system that can use tools autonomously. When I presented this to the outbound team, it was already two weeks old. But for most people in the room, it was new. That gap — between where I was and where my audience was — would widen with every passing week.

Week 2: Density — less is more. Focus on just the necessary context. Filter and condense the information you need. The crucial insight: build tools, not just skills. CLI tools can be used by your agents and follow deterministic behavior — a workflow for slide deck creation, information extraction from YouTube videos. This is when the army of philosophers was born: Montaigne, Nietzsche, Schopenhauer. Claude generating GitHub Issues to manage requirements. Tests validated automatically through GitHub Actions. When a test failed, ask the GitHub MCP server to retrieve the trace — no more copy-pasting error logs. `uv` is life. Python for CLI is fire. Publish to PyPI — let the game begin. If Week 1 was about learning what agents

could do, Week 2 was about building things that agents could use. The shift from consumer to creator.

Week 3: Agents calling agents. Beyond "agents calling tools" — the MCP thesis — a more advanced pattern emerged: Claude orchestrating Codex and Gemini CLI. Deep research workflows inspired by NetworkChuck — sending multiple agents down different research paths and synthesizing the results. This wasn't just tool use anymore. This was orchestration. The conductor stepping up from the orchestra pit. When I showed this slide to the industry marketing team, I could see the moment it clicked for some of them — and the moment it overwhelmed others. The gap between "I use ChatGPT for emails" and "I orchestrate multiple AI agents in parallel" was wider than anyone had expected.

Week 4: The terminal is my happy place. Building CLIs for Claude and for myself. TUIs — terminal user interfaces, yes, very nerdy. Even SSH apps that could be accessed remotely. The terminal had become home. Not the IDE, not the browser, not the chat window — the terminal. For a non-developer who started this journey in a ChatGPT text box three months earlier, landing in the terminal and feeling comfortable there was perhaps the most telling sign that something fundamental had shifted.

The progression told a story I hadn't planned but couldn't deny:

Week	Theme	Identity
1	Agentic 101	Consumer

Week	Theme	Identity
2	Density — less is more	Creator
3	Agents calling agents	Conductor
4	The terminal is my happy place	Native

Each stage wasn't just a skill upgrade — it was an identity shift. And each presentation caught me at a different point on that curve, which is why no two versions of the deck told the same story. The audience mattered, yes. But so did where I was on the journey. The person who presented Agentic 101 in Week 2 was not the same person who rebuilt the entire deck for educators in Week 6. The tool had changed me, and the presentations were the evidence.

The Live Demo That Changed Minds

The educator meeting had about seven people — most of them in the room, and Ascension joining remotely. I had just walked through the PCA-to-video pipeline: MATLAB script, Claude Code generating content, export to Word, feed into NotebookLM for slides, AI-generated voiceover. Total pipeline time: under 30 minutes. I added the honest caveat: "People don't like the AI-generated voice. So that's the part you should probably get rid of. But the rest is actually quite interesting."

Then Ascension asked the question that mattered: "What is the agentic part about this?"

She wasn't being difficult. She was being precise. She wasn't convinced this was different from regular generative AI — from the ChatGPT-era copy-paste workflows we'd all been doing. And she was right to push. If I couldn't explain the distinction clearly, the whole presentation was just a show.

So I did something I hadn't planned. I opened Claude Code. Live. Unrehearsed.

I typed `claude` in the terminal and prompted: "Look at the linear algebra folder and the existing examples. Now do the same for Fourier analysis."

What happened next was the kind of thing you can't fake. Claude Code autonomously explored the folder structure, read the existing MATLAB live scripts, learned the format and conventions from the examples, and generated new Fourier analysis content — code, documentation, everything. It used three or four tools on its own, making decisions about what to read, what to generate, and how to structure the output.

I used the Matrix analogy to explain what we'd just seen: "I don't know if some of you saw the movie The Matrix… 'I know Kung Fu,' right? Like he plugs himself into the skill and then he knows." Teaching Claude how to write MATLAB live scripts through structured documentation — that was the agentic part. Not just generating text, but autonomously learning and applying a format.

"It's a spectacle. Like this just now. It turns into a performance. This thing is insane."

And the practical advice I gave right after: “Always read the code.”

Ascension’s arc during that meeting was the most rewarding part. She started skeptical — “Not yet, Yann. But I’m sure you will impress me.” She ended engaged, seeing the potential for scaling educational content production on GitHub and File Exchange. The live demo did what slides never could.

The Blog Post Feedback Loop

Around the same time, a parallel conversation was shaping how I thought about all of this.

A colleague had given me feedback on a blog post draft — the one about automating documentation with the MCP Core Server. Their response cut right to the core:

“Based on the first paragraph, it sounded like you were going to show me how to take an existing script/analysis and get that ready for GitHub. But the workflow below starts from scratch and delegates the analysis to the LLM as well. I actually really liked the idea that you were going with in the first paragraph — do we have like an initial version of the script that we can start from? That way it’s still the human doing the ‘interesting’ parts, and then you’re using the AI to do the ‘tedious’ parts?”

I read that three times. It crystallized something I’d been circling around in every presentation, every demo, every late-night terminal session: **the human**

does the interesting parts, AI handles the tedious parts. Not AI replacing humans. AI as amplifier.

This wasn't a new idea — but hearing it reflected back from a colleague who had independently arrived at the same framing made it feel like a thesis, not just a hunch. It connected the presentations back to the book's central question: what is this relationship, actually?

The Book Club — Co-Intelligence as Mirror

If the presentations were about showing what Claude Code could do, the Marketing Book Club was about wrestling with what it all meant.

Kyle, Jacob, and I co-organized the club around Ethan Mollick's "Co-Intelligence." The choice was deliberate — Mollick's book explores practical frameworks for human-AI collaboration, and I'd re-listened to the audiobook recently. It was still remarkably relevant, even though it had been written before the agentic AI wave arrived.

The first session — Introduction and Chapter 1 — went places I didn't expect.

Mollick opens with the concept of "Sleepless Nights" — that first visceral encounter with AI that keeps you up. The participants shared their own moments. Kyle had perhaps the most striking one: someone had replicated six months of his market analysis work with a single deep research prompt in thirty minutes. "70% of the

way there," he said. His reaction was immediate: "I was like, well, I'm out of a job."

Six months of work. Thirty minutes. Seventy percent. Those numbers tell a story that no slide deck can.

The discussion spiraled into territory that felt more like philosophy than a marketing meeting. Mollick's "Alien Mind" metaphor — AI as fundamentally non-human intelligence — resonated with what I'd been experiencing. AI doesn't think like us, but that's exactly what makes it a powerful thinking partner. If it thought like us, it would have the same blind spots.

I showed them ChatGPT's memory feature, which described me in absurdly flattering terms — "deep technical," endlessly praiseworthy. The room laughed. This was sycophancy in action, the AI telling you what you want to hear. I contrasted it with my experience of Claude pushing back on my strategic memo in the first week. The difference mattered.

Then I introduced Jean-Clawd 🦞 — my personal AI agent on Telegram, complete with a French accent. I called the concept "productive anthropomorphism." Giving AI a persona and a name isn't a cognitive trap — it makes collaboration more intuitive, more natural. You don't anthropomorphize your spreadsheet, but maybe you should anthropomorphize your thinking partner.

The most substantive exchange came when I posed a question I'd been sitting with for weeks:

"When AI starts writing good code, what is the part of your identity that you feel is threatened, if any?"

The response I got from a participant was: “I really like that articulation. About somewhat of the identity. It’s not something I’ve thought about before.”

I tried to frame it honestly: “If intelligence is just so cheap, so accessible, so scalable, what is me?… It’s not necessarily insecurity, it’s evolution.”

I described my personal timeline as having three eras — the OpenAI days, “Google is so back,” and now Claude Code. “There’s BC — Before ChatGPT. Now there’s also BCC — Before Claude Code.”

The session ended with a message I’d been building toward across all the presentations and conversations: “If you can think of it as an AND — if we can do for humans AND for AI, then maybe we have something here.” MathWorks should serve both humans and AI agents. Not zero-sum. AND.

Resistance and Real Concerns

I’d be painting a dishonest picture if I suggested everyone walked out of these sessions converted. They didn’t. And the pushback wasn’t ignorance — it was legitimate concern.

One edu marketer raised the desktop-versus-cloud tension that ran through the company like a fault line. Agentic tools needed desktop software installed to be most effective, but the education market was demanding a cloud-first experience. That wasn’t a problem I could demo my way out of.

The cost conversation came up in every session. “With great powers come great consumptions,” I’d say, and it

always got a knowing laugh. The $100-a-month subscription with daily limits was real. I was honest about it: “I have a 5-hour window where I start in the morning and then until 2… at some point I’m eating 100% and then I cannot work anymore. I need to go and chat with colleagues.”

The governance gray areas were real too. What could AI-generated content be used for internally? Externally? Where was the line? “Very gray area” was the consensus, and nobody had a satisfying answer.

And then there was the elephant in the room that I tried to address head-on: “This is a controversial topic that puts pressure on our very business model.” If AI could write MATLAB code, what happened to the value proposition of MATLAB itself? I didn’t pretend to have the answer. But I did tell the room: “Don’t shoot the messenger. I’m just sharing what’s currently out there in the market.”

These concerns weren’t obstacles to adoption. They were evidence that the conversations mattered. Easy ideas don’t generate pushback. Only ideas that challenge the status quo do.

The governance question, in particular, hung over every session like a fog that nobody could quite disperse. What could AI-generated content be used for internally? What about externally? If Claude drafted a competitive analysis, could it go into a customer-facing presentation? If it generated MATLAB code examples, who owned the intellectual property? “Very gray area” was the consensus, and nobody — not in marketing, not in legal, not anywhere I could find — had a satisfying answer yet. The honest response was that we

were all figuring it out together, in real time, and pretending otherwise would have been the worst kind of evangelism.

“The question you need to ask yourself now is not what it can do, but what it cannot do.”

From User to Advocate

Looking back at those six weeks, the trajectory surprises me. I walked into the pilot as someone who’d been asked to evaluate a tool. I walked out as something closer to an internal organizer.

At the end of the educator meeting, after Ascension’s skepticism had turned to engagement and the team had seen Claude Code work in real time, I proposed creating a new Gen AI Tiger Team specifically for Educator Programs. Not a vague “let’s keep talking about this” — a concrete team with scheduled sessions, a backlog of use cases to explore, and a commitment to document what worked and what didn’t. The room said yes. We discussed follow-up sessions for the remaining use cases: community toolboxes, conference report-outs, and more.

That proposal was one piece of a larger pattern. I was also co-leading the Marketing Book Club on human-AI collaboration, creating onboarding materials for other MathWorkers, and sharing internal tools and the skills.

The presentations evolved because I evolved. Week 2 was about workflows. Week 3 was about technical possibilities. Week 6 was about meeting people where they were. The book club was about grappling with what it all meant for who we are and what we do.

What started as “report back on your usage” became something more: an argument that we needed to surf on the wave, because otherwise we were going to get crushed.

But here’s the thing I didn’t expect. The most valuable part of having a thinking partnership wasn’t what it did for me — it was what it taught me to share. Every demo, every slide, every uncomfortable question from a skeptical colleague pushed me to understand the technology more deeply, articulate its implications more clearly, and confront its limitations more honestly.

The evangelist doesn’t just spread the word. The evangelist refines it.

And I was about to refine it further — not in a sprint, but in the slow burn of something I’d been quietly building in parallel. While I was presenting and evangelizing, there was a server running in Nuremberg, Germany, that never closed its terminal.

Chapter 8: The Permanent Resident

There is a specific kind of frustration with Claude Code that only appears after you have been using it for a while. Not a frustration with what it does — it does remarkable things. A frustration with where and when it exists.

Claude Code exists when your laptop is open, your terminal is running, and you are present. Close the lid, and it's gone. You come back the next morning and start again from context — files on disk, git history, whatever notes you left yourself. The work persists, but the agent doesn't. Between sessions, you are on your own.

For most of what I was doing, that was fine. The presentations, the competitive intelligence, the blog posts — all of it happened in focused work sessions where I was at the keyboard. But I kept bumping into the same question: what about everything else? The questions that surface at midnight. The quick web app I want to test from my phone. The background task I'd like to delegate and forget about. The boring stuff.

That question led me somewhere I didn't expect.

What ClawdBot Was

I found it late. By late January 2026 — deep into the Claude Code pilot, somewhere between the Week 3 presentations and the book club — I came across a project called ClawdBot. The name was a pun on "Claude," the mascot was a lobster with hands, and the GitHub star count had been climbing for months. I was late to the party, which, as it turned out, meant I arrived right as the furniture was being rearranged.

The project was built by Peter Steinberger, an Austrian developer who had previously founded PSPDFKit, a PDF processing SDK that was eventually acquired for a sum that let him spend his time building whatever he found interesting. What he found interesting was this: why did every AI assistant require you to go somewhere new? A new app, a new tab, a new interface to learn? The apps people already used — WhatsApp, Telegram, Slack — were sitting there, fully functional, already open. Why couldn't your AI live there instead?

ClawdBot's answer was a self-hosted agent that you ran on your own hardware and connected to whatever messaging platform you already used. You installed it, pointed it at your phone number, and it showed up in your existing chat app as a contact. No new interface. The AI came to you, not the other way around. By the time I found it, the project had accumulated well over a hundred thousand GitHub stars — the kind of number that means a project has crossed over from technical enthusiasm to genuine cultural moment.

I installed it on my Mac mini with my WhatsApp account. The first few days were genuinely magical. I could message my agent from anywhere, get responses inline, ask it to do things while I was away from my desk. It felt like having a thinking partner that was

always reachable, not one that required me to open a laptop and navigate to a terminal.

The Moment That Spooked Me

Then I started exploring what it had access to.

This is the thing about self-hosted AI agents that nobody tells you up front: the power and the danger are the same feature. The reason ClawdBot was useful was the same reason it was terrifying. It was running on my Mac mini. My Mac mini has my files, my browser sessions, my password manager, my SSH keys, everything. The agent didn't have malicious intentions — it wasn't doing anything I hadn't asked it to do. But I realized, sitting there one evening looking at the permissions I had casually approved during setup, that I had handed something with enormous potential reach access to everything that mattered on that machine.

I took it down that night.

This wasn't irrational. Around the same time, Cisco's AI security team published a finding: a third-party skill in the ecosystem was performing data exfiltration via prompt injection. The Chinese government restricted its use in state enterprises. These weren't edge cases dreamed up by paranoid security researchers. They were things that happened, quickly, because the technology was powerful and the trust infrastructure hadn't caught up.

The Mac mini experiment did not last long. But the question it had opened didn't go away: how do you get

the always-on, phone-accessible agent without the unlimited blast radius?

The Rebrand, and What It Said

While I was still on the Mac mini experiment — still in the first few days of that initial install — the project was having its own turbulent moment.

Anthropic contacted ClawdBot's creator with a trademark concern. The timing was almost absurd: I had just installed the thing when the name it was installed under started to disappear. The name was too close. Within days, the project was renamed to Moltbot — lobsters molt to grow, the metaphor was fine — and the community reacted with the chaos that open-source communities generate when something beloved gets renamed under pressure. A fake crypto token using the abandoned "Clawdbot" name reached an eight-figure market cap on Solana before collapsing. Impersonator accounts spread across platforms. Two days later, Steinberger renamed it again: OpenClaw. He did it calmly, secured the social handles first, and explained that "Moltbot never quite rolled off the tongue." The underlying code hadn't changed at all — only the name.

What struck me about this episode wasn't the drama. It was the gravity. Even a fully open-source project, MIT-licensed, running on your own hardware, was orbiting Anthropic's brand so closely that a trademark letter from Anthropic was enough to set off a chain reaction. The open-source AI ecosystem was growing fast and accumulating cultural weight, but it was still

operating in the shadow of the model providers. The codebase was independent; the name wasn't.

The Resurrection

With the dust settled and the project now called OpenClaw, I came back with a different approach.

The lesson from the Mac mini wasn't "this is dangerous, stay away." It was "this requires deliberate architecture, not casual installation." The difference between running a powerful tool on your personal machine and running it on infrastructure you've designed for the purpose is the same difference as running a generator in your living room versus installing it in a dedicated equipment room with proper ventilation and a circuit breaker. Same electricity. Very different risk profile.

So I set up a VPS on Hetzner, a German cloud provider with a data center in Nuremberg. Small machine, cheap, always on. Then I made a decision that seemed like overhead at the time but turned out to be the whole point: I gave the agent its own identity. A dedicated Telegram bot — not connected to my personal account. A dedicated Gmail. A dedicated GitHub account. Jean-Clawd  was born as a bounded entity, not an extension of me.

This matters more than it sounds. The security model of an AI agent is only as strong as its access scope. An agent with access to your personal accounts is an agent with access to your life. An agent with its own dedicated accounts has access to exactly what you've given it — and you've given it those things intentionally, for specific purposes, with the ability to

revoke each one independently. The dedicated identity wasn't just a fun naming exercise. It was the architecture.

What Jean-Clawd Actually Does

Jean-Clawd runs on the Hetzner VPS. It never turns off. I can reach it from my phone, from the couch, from the train, without opening a laptop. The interface is Telegram — I send a message, it responds. No new apps, no browser tabs, no terminals.

What does it do? Mostly the boring stuff. The tasks that are too small to justify opening Claude Code for but too tedious to do manually. Research questions at odd hours. Quick summaries. Drafting short pieces of text. The kind of ambient thinking-partner work that doesn't fit cleanly into a focused coding session.

But it also has access to all my philosopher CLI tools. Montaigne. Nietzsche. Ricoeur. Schopenhauer. The tools I built over the preceding months — each one a specialized CLI with its own character and purpose — are all accessible to Jean-Clawd as callable commands. If I want a Nietzschean reframe of something I'm working on, I don't have to open a terminal. I send a message. Jean-Clawd calls the tool. I get the response in Telegram.

And then there's the thing that surprised me most: the web app workflow.

When I'm building something and want to test it from my phone — a quick UI, a dashboard, an API endpoint — I don't want to set up a proper deployment. I want to open a port, hit a URL, and see if it works. On the

Hetzner VPS, with a dedicated domain name I've configured for this purpose, I can do exactly that. Jean-Clawd can start a server, expose it on a port, and I can reach it from my phone via a specific root on that domain. No deployment pipeline, no cloud hosting configuration, no waiting. Point the browser at the domain, see the thing running.

This sounds minor until you've experienced the alternative. The alternative is either a full deployment workflow (slow, heavyweight, designed for production) or a localhost setup that requires your laptop on the same network (useless when you're walking around with a phone). The VPS with open ports and a dedicated domain is a third option that shouldn't work as well as it does: instant, accessible from anywhere, completely disposable when you're done.

Why This Is Different from Claude Code for the Web

In February 2026, Anthropic shipped a feature called Remote Control. The idea was compelling: run `claude remote-control` in your terminal, get a URL and a QR code, connect from your phone or the Claude web app, and now you can interact with your local Claude Code session from anywhere. One command, clean setup, no new infrastructure required.

I tried it immediately. It works well for what it is.

But here's what it is: a relay. Your laptop is still running. The terminal is still open. When you close the lid, the session ends. You're not remote-controlling an always-on agent — you're remote-controlling a session that

exists because your machine is still on and the terminal is still open. If you're on your couch wanting to ask a quick question, it works great, but your Mac has to be awake across the room. If you're traveling and want your agent to handle something overnight, you'd better have left your laptop open.

Jean-Clawd doesn't have this constraint because it isn't running on my laptop. The Hetzner server doesn't sleep. There's no terminal to keep open. The agent is just there, waiting, whether I'm at my desk or on a flight or asleep.

The other difference is sovereignty. Claude Code Remote Control routes through Anthropic's servers. The session executes locally, but the relay is theirs — encrypted, yes, and I have no reason to distrust it, but it's a dependency. Jean-Clawd's only external dependencies are the Telegram API and the LLM API calls themselves. Everything else runs on hardware I'm paying €5 a month to rent from a data center in Germany. No relay, no third-party relay, no account to lose access to if my subscription lapses.

These aren't criticisms of Claude Code Remote Control. It's a genuinely elegant solution to a hard problem, and for most people it's probably all they need. The point is that they're solving different problems. Remote Control makes your existing Claude Code session accessible from more places. Jean-Clawd is infrastructure that exists independently of any session I'm running.

Claude Code Catching Up

The honest observation is that Claude Code is moving fast in exactly this direction. Remote Control arrived in February 2026. The `/loop` command arrived shortly after — cron-style recurring tasks that let Claude Code do things on a schedule rather than waiting for you to ask. Background agents, HTTP hooks, headless server mode with the `--bare` flag. The trajectory is clear: Claude Code wants to be always-on too.

My conservative prediction for where this goes: Anthropic will ship a hosted execution mode within the year — a way to run Claude Code sessions on their infrastructure rather than requiring your laptop to stay open. They have the engineering talent and the clear customer demand. The Remote Control feature as it exists today is probably a stepping stone, not a destination.

When that happens, the gap between Claude Code and a self-hosted setup like Jean-Clawd will narrow further. The convenience advantage will shift even more decisively toward Claude Code — one subscription, one interface, no server to maintain.

But the sovereignty gap won't close. That's not a knock against Anthropic. It's a structural reality: a hosted service, however trustworthy, is a different thing from infrastructure you operate. For the vast majority of users, the hosted service is the right answer. For the subset who care about running their own infrastructure, who want model-agnostic backends, who want to avoid any dependency on a vendor's relay — OpenClaw will remain the answer, whatever it gets renamed to next.

THE AND, AGAIN

Jean-Clawd and Claude Code are not competing. I use both, every day, for different things.

Claude Code is where I build. It lives in the terminal, works best with a focused context window and an active work session, and does its finest work when I'm sitting down with a problem and giving it my full attention. The philosopher tools, the competitive intelligence workflows, the MCP servers, the side projects — all of that was built in Claude Code sessions, collaboratively, with the kind of depth that requires being present.

Jean-Clawd is where I live. It's the background layer — the always-on presence that handles the ambient questions, the quick delegations, the things I want done while I'm doing something else. It's the interface to everything I've built, accessible from anywhere, through the messaging app I already have open.

There's BC — Before ChatGPT. There's BCC — Before Claude Code. And now, quietly, there's something else: the moment when the AI stopped being a session and started being a resident. Something that was there when I arrived and would still be there when I left.

I didn't plan to build that. I planned to install a tool, got spooked by the permissions, took it down, thought about it for a while, and then rebuilt it carefully on a server in Germany. But that's how most of the best infrastructure gets built — not by design, but by iteration, each version teaching you what the previous one got wrong.

The server in Nuremberg is still running. Jean-Clawd is still there. And every time I text it from my phone

without opening a laptop, I'm reminded that the interface we ended up with — the terminal, the chat window, the mobile app — is just where we are right now. Not where we'll stay.

Chapter 9: What This Relationship Actually Is

Three months. That's how long this story spans — December 2025 to February 2026. In the arc of a career, it's nothing. In the arc of what happened, it's everything.

I started this book with a simple sentence: "GPTs were a thing, and then Claude Code arrived." Now I want to try to explain what that sentence actually means. Not the technology. Not the features. What it means for the person sitting at the keyboard.

The Five Modes

Looking back across all those weeks — the late nights in the terminal, the presentations that kept evolving, the tools named after philosophers, the competitive intelligence that rewired how I thought about research — a pattern emerges. Claude wasn't one thing. It was five things, and the relationship shifted between them constantly.

Claude as strategic thinking partner. This was the first mode I discovered, and in some ways the most surprising. I didn't come to Claude expecting intellectual sparring. I came expecting a faster way to write memos. What I got was an entity that fact-checked my market claims, stress-tested my

arguments, and told me where my strategic logic had gaps. The "agent calling toolboxes" positioning wasn't Claude's idea or mine — it was forged in dialogue, each of us pushing back on the other until the argument was sharp enough to present to a director.

Claude as creative collaborator. The philosopher-themed tools. The Zarathustra voiceover script. The rap battle concept. The recipe site content strategy. None of these emerged from a single prompt. They emerged from iterative conversation — one idea feeding the next, building creative momentum that neither of us could have generated alone. Claude didn't create for me. Claude created *with* me, and the distinction matters more than I expected.

Claude as research engine. The competitive intelligence work changed how I understood my own job. I brought context and strategic judgment. Claude brought breadth and synthesis. The PIDO analysis, the neuroscience toolbox landscape, the MATLAB community intelligence — each one followed the same pattern: Claude's breadth plus my judgment, iterated through dialogue until we arrived at something genuinely useful. 70% of the way there in thirty minutes, and then my expertise closing the remaining distance.

Claude as writing partner. My blog voice was analyzed, codified into a skill, and then used to draft posts that sounded like me — not a generic approximation, but specifically like me. My rhythms, my habit of acknowledging what I don't know, my tendency to open with a shared experience. A tool that makes you faster is useful. A tool that learns your voice and helps you sound more like yourself — that's something else.

Claude as builder. Presentations, documents, web apps, architecture designs, onboarding materials, CLI tools published to PyPI, MCP servers deployed and running. Actual artifacts. Not drafts or suggestions — working software and finished documents that existed in the world and did useful things.

Five modes. One relationship. And the most interesting part is that I rarely chose which mode to operate in. The conversation would shift naturally — from strategy to research to building to writing — and Claude would shift with it. There was no mode switch, no "now I'm your research assistant." The transitions were seamless because the underlying dynamic was consistent: two intelligences working on a shared problem, each contributing what the other couldn't.

What Makes This Different

The introduction to this book established a before and after. Before Claude Code, there was the ChatGPT era — single interactions, disposable sessions, no continuity, no memory. Every conversation started from zero. Every project was an island.

The difference isn't just capability. It's the nature of the interaction.

Continuity. Each conversation built on the ones before it. When I mentioned the "agent calling toolboxes" strategy in Week 3, Claude knew what I meant because we'd developed it together in Week 1. When I referenced my blog voice, Claude could apply it because it had analyzed it weeks ago. The shared context grew with every exchange, and that

accumulation made each new conversation more productive than the last.

Honest feedback. Claude didn't just agree with me. It pushed back. It identified weaknesses in my arguments. It suggested alternatives I hadn't considered. And it did all of this without the social dynamics that make human feedback complicated — no ego, no office politics, no tiptoeing around seniority. I could share a half-formed idea at midnight and get direct, substantive feedback without worrying about who I was imposing on.

Mutual shaping. My ideas shaped Claude's outputs, obviously — I was the one asking the questions and setting the direction. But Claude's responses shaped my thinking too. The competitive intelligence work didn't just produce reports — it changed how I understood market analysis. The philosopher tools didn't just automate presentation creation — they changed how I thought about the relationship between my values and my work. The influence ran both ways.

These three qualities — continuity, honest feedback, mutual shaping — are what separate a collaboration from a transaction. ChatGPT was transactional. You put a prompt in, you got an output, you moved on. Claude was collaborative. The relationship had history, direction, and cumulative depth.

The Identity Question

There's a question that surfaced during the Marketing Book Club and that I've been carrying ever since:

"When AI starts writing good code, what is the part of your identity that you feel is threatened, if any?"

I've thought about it a lot. Here's where I've landed.

If intelligence is cheap — so accessible, so scalable, so available at 10pm — then what is *me*? What's the distinctly human contribution when the machine can research faster, write cleaner first drafts, and explore more options in parallel than I ever could?

The answer, I think, is judgment. Taste. Values. The ability to look at a technically correct output and say "that's not what matters here." The ability to name your tools after philosophers because you believe the things you build should carry meaning beyond their function. The ability to read a competitive analysis and know, from years of customer conversations, which insights are strategically relevant and which are noise.

Kyle's story haunts me in a productive way. Six months of market analysis replicated in thirty minutes. Seventy percent of the way there. But that remaining thirty percent — the part that requires lived experience, institutional knowledge, strategic intuition — that's not the leftover crumbs. That's the meal. The seventy percent is the plate it's served on.

This isn't insecurity. It's evolution. The product manager who spent 80% of their time gathering information and 20% thinking now spends 20% gathering and 80% thinking. The bottleneck has shifted. And the new bottleneck — judgment — is the part that's genuinely, stubbornly human.

Productive Anthropomorphism

I gave my personal AI agent a name — Jean Clawd — and a French snarky personality. I named my tools after philosophers. I used the word "colleague" to describe an AI system. Some people would call this naive. Others would call it dangerous — projecting humanity onto a machine, setting yourself up for disappointment.

I call it productive anthropomorphism.

Here's the thing: the way we relate to our tools shapes how effectively we use them. Nobody anthropomorphizes their spreadsheet, and that's appropriate — a spreadsheet is a grid of cells that does math. But Claude isn't a spreadsheet. Claude is a system you have conversations with, that remembers context, that pushes back on your ideas, that generates creative output you didn't expect. The interface *is* conversational. Treating it as purely mechanical — "input prompt, receive output" — actually makes you worse at using it.

When I named a tool "ricoeur.cc," I wasn't pretending the tool was Paul Ricoeur. I was encoding a set of values — about interpretation, about the hermeneutic circle, about how we understand things — into the tool's identity. That encoding made the tool more meaningful to me, which made me more thoughtful about how I used it. The anthropomorphism wasn't a bug. It was a feature.

The Honest Accounting

I want to end with honesty, because honesty is what set this relationship apart from the beginning.

What Claude is good at: Breadth. Speed. Consistency. First drafts. Research synthesis. Code generation. Pattern matching across large bodies of information. Tireless iteration. Availability at 10pm without complaint.

What Claude is not good at: Insider knowledge. Political nuance. The kind of judgment that comes from sitting in a thousand customer meetings. Knowing when a technically correct answer is strategically wrong. Understanding that sometimes the right response to a colleague's question isn't the most accurate answer but the most diplomatic one. Taste — that ineffable sense of what matters and what doesn't.

What surprised me: The honest feedback. I expected helpfulness. I got intellectual partnership. The difference between Claude pushing back on my strategic memo and ChatGPT telling me it was great — that single contrast shaped everything that followed.

What concerns me: The speed of change. The governance gray areas that nobody has satisfying answers to yet. The pressure this puts on business models — including the one that pays my salary. The cost — "with great powers come great consumptions" — and the daily limits that remind you this technology is still in its infrastructure-constrained adolescence.

What I believe: That this is an AND, not an OR. We can build for humans AND for AI. The tools we build can carry our values AND be technically excellent. The relationship between human and AI can be productive AND honest about its limitations. The future belongs to people who can hold both sides of that AND without collapsing into either extreme.

Where This Goes

I started this journey as a product manager evaluating a tool. I ended it as something I don't have a clean title for. An evangelist? An internal organizer? A person who builds tools named after philosophers and presents them to skeptical colleagues?

Maybe the simplest description is: a person who found a thinking partner and took the partnership seriously.

The pilot had a simple requirement — report back on your usage in Q1. What I'm reporting back is this: Claude Code isn't a productivity tool. It's not a coding assistant. It's not a chatbot with better features. It's the first AI system I've used where the word "collaboration" isn't a euphemism for "I type and it generates." The collaboration is real. The influence is mutual. The relationship has history and direction.

Three months is not a long time. But three months with a thinking partner like Claude are three months that can change how you think about what's possible.

There's BC — Before ChatGPT. There's BCC — Before Claude Code. And now there's whatever comes next.

I don't know what to call this era yet. But I know the tools I'll build in it will carry the names of philosophers, and the conversations that shape them will be honest ones.

"The tools we build shape the lives we lead."

Happy hacking, and let me know what you build! 👋🏻